

Creating Small and Efficient Networks

Chen-Change Loy

吕健勤

<http://www.ntu.edu.sg/home/ccloy/>

<https://twitter.com/ccloy>



Credits

- Deep Compression

- http://web.eng.tau.ac.il/deep_learn/wp-content/uploads/2016/12/deep_compression.pptx

- Dark Knowledge

- <https://www.ttic.edu/dl/dark14.pdf>

Outline

- Network pruning
- Knowledge distillation

Motivation

The huge **memory footprint** and lengthy **inference time** have made cumbersome networks prohibitive from deployment in many **resource-limited** mobile systems and edge devices.

Model	Parameter
LeNet-5	1 M
AlexNet	240 M
VGG-16	552 M



Network Pruning

Motivation

To alleviate the demand on memory storage and speed up the model inference, we hope to remove some **unimportant** parameters of the original network without incurring severe performance degradation.

Three objectives of network pruning:

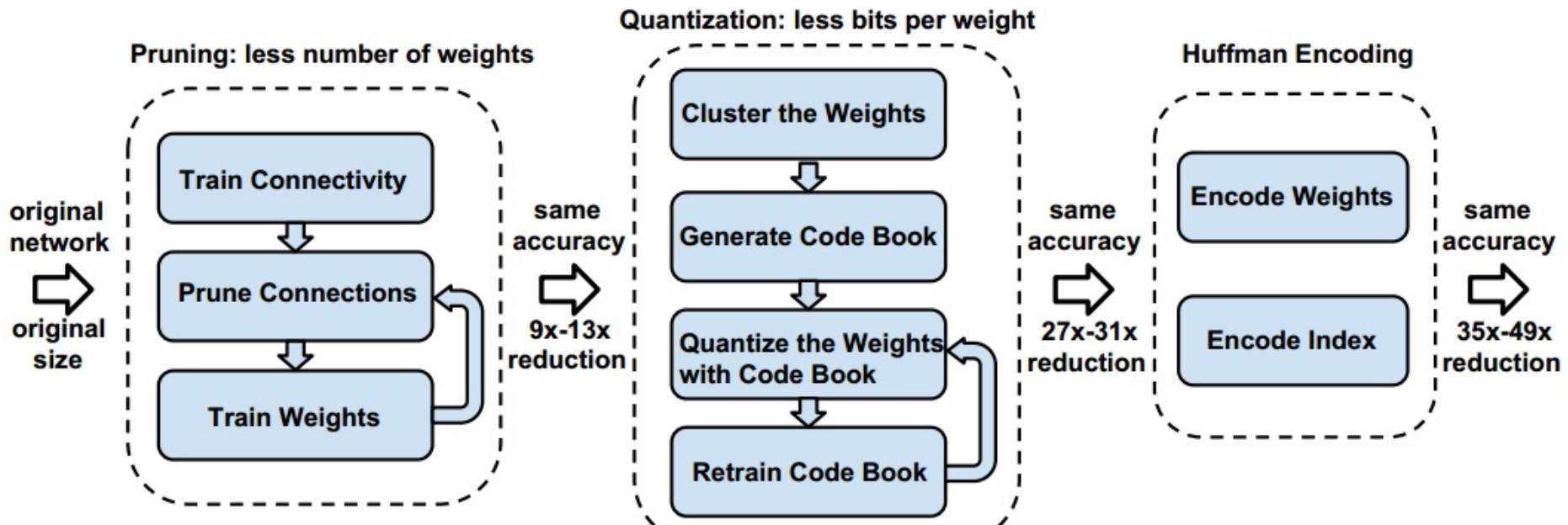
- Reduce model parameters
- Reduce model latency
- Retain model performance

Network pruning

- General steps
 - Training a large overparameterized network
 - Pruning the less-important weights or channels
 - Finetuning or re-training the pruned network

Seminal Work: Deep Compression

Han et al. propose deep compression, which is comprised of magnitude-based network pruning, quantization and Huffman encoding.



Magnitude-based Network Pruning

Motivation:

Network weights whose magnitudes are close to zero are considered less important and trimming these weights will not incur severe accuracy drop.

Method:

- 1. Choose a neural network architecture.
- 2. Train the network until a reasonable solution is obtained.
- 3. Prune the weights of which magnitudes are less than a threshold τ .
- 4. Train the network until a reasonable solution is obtained.
- 5. Iterate to step 3.

0.8	0.3	-0.1
0.6	0.15	0.4
-0.7	0.12	-0.2

Prune parameters
whose **absolute value**
is smaller than 0.2



0.8	0.3	0
0.6	0	0.4
-0.7	0	-0.2

Magnitude-based Network Pruning

Results:

Network	Top-1 Error	Top-5 Error	Parameters	Compression Rate	
LeNet-300-100 Ref	1.64%	-	267K		}
LeNet-300-100 Pruned	1.59%	-	22K	12×	
LeNet-5 Ref	0.80%	-	431K		
LeNet-5 Pruned	0.77%	-	36K	12×	
AlexNet Ref	42.78%	19.73%	61M		}
AlexNet Pruned	42.77%	19.67%	6.7M	9×	
VGG-16 Ref	31.50%	11.32%	138M		
VGG-16 Pruned	31.34%	10.88%	10.3M	13×	

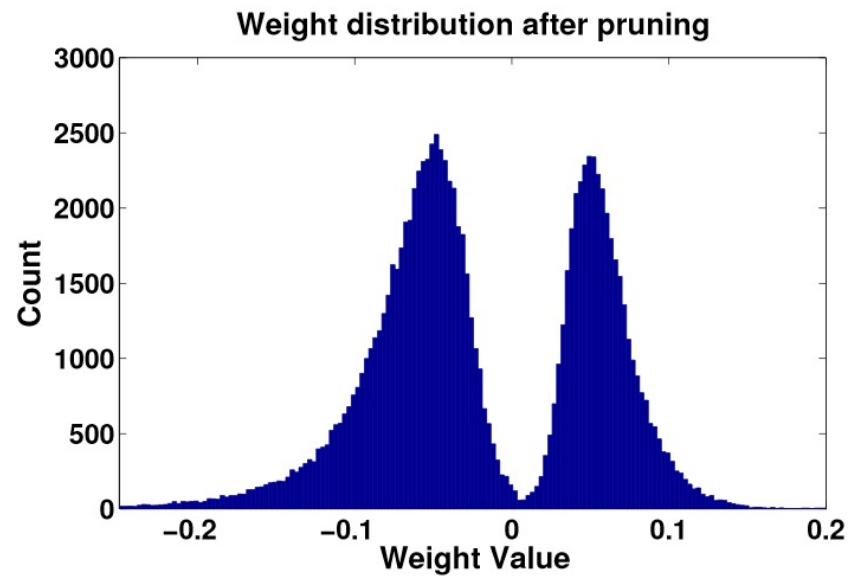
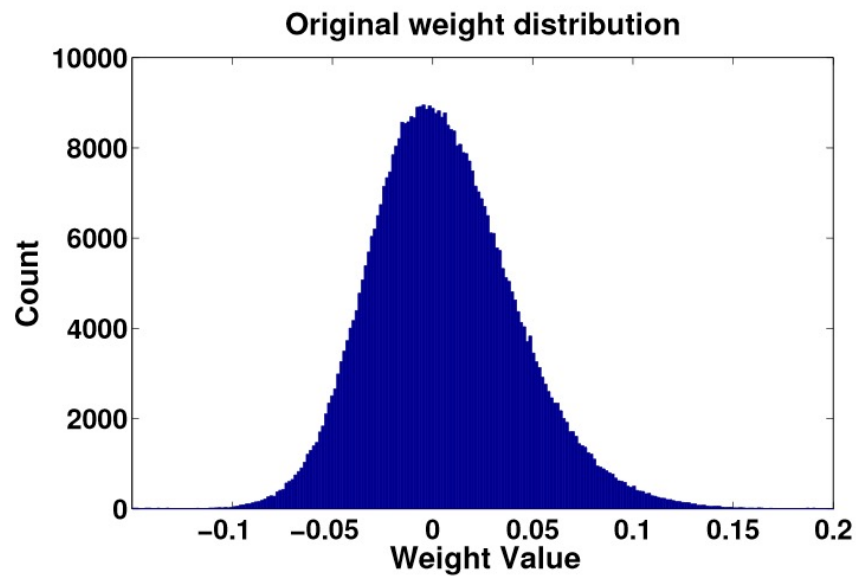
MNIST

ImageNet

The proposed pruning algorithm can save 9× to 13× parameter storage with almost no loss of accuracy.

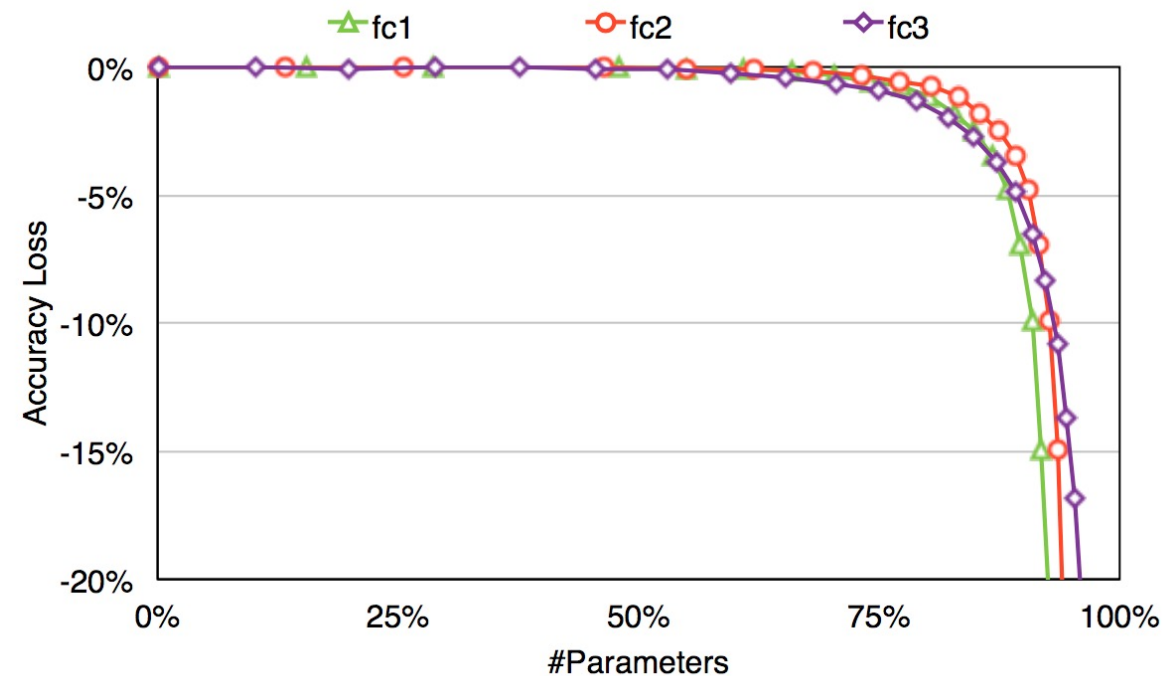
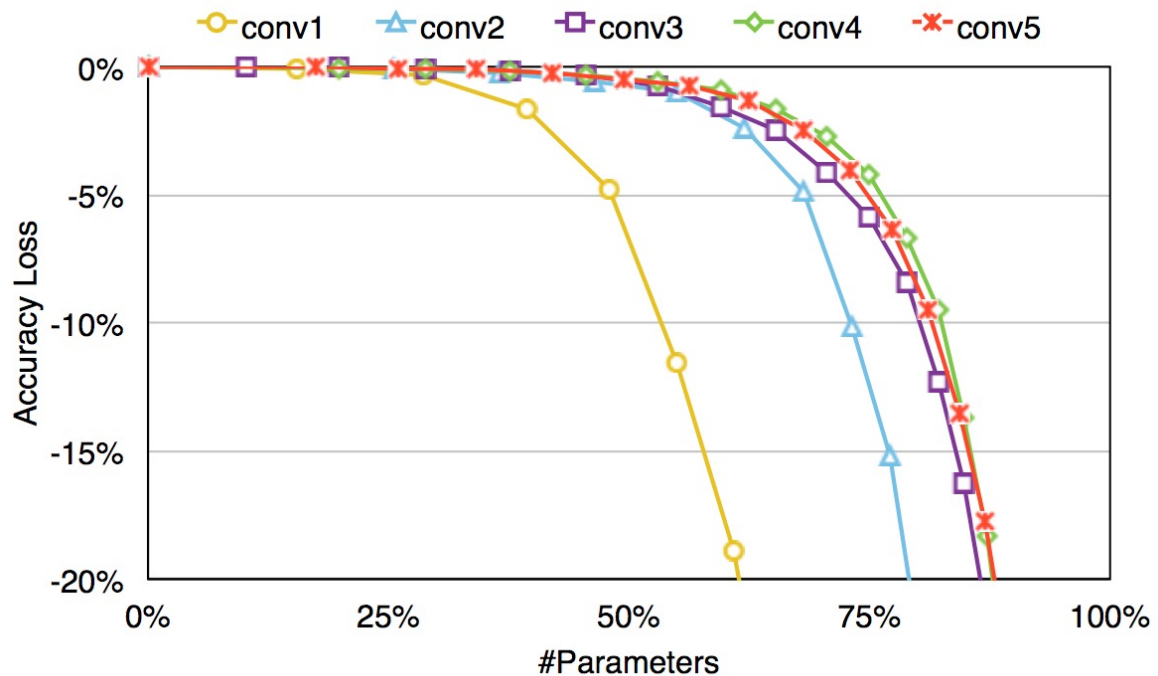
Magnitude-based Network Pruning

Weight distribution:



After pruning and retraining, parameters with small magnitude are greatly reduced.

Magnitude-based Network Pruning



Magnitude-based Network Pruning

Issues:

- Pruned connections have no chance to come back.
- Incorrect pruning may cause severe accuracy loss.

How to avoid the risk of irretrievable network damage?

Dynamic Network Surgery

W_k denotes the weights, and T_k denotes the corresponding 0/1 masks.

$$\min_{W_k, T_k} L(W_k \odot T_k) \quad s. t. \quad T_k^{(i,j)} = h_k(W_k^{(i,j)}), \forall (i,j) \in \mathfrak{I}$$

$\odot$ is the element-wise product. $L(\cdot)$ is the loss function.

Dynamic network surgery updates only W_k . T_k is updated based on $h_k(\cdot)$

$$h_k(W_k^{(i,j)}) = \begin{cases} 0 & a_k \geq |W_k^{(i,j)}| \\ T_k^{(i,j)} & a_k \leq |W_k^{(i,j)}| \leq b_k \\ 1 & b_k \leq |W_k^{(i,j)}| \end{cases}$$

$h_k(\cdot)$ is a discriminative function, which satisfies $h_k(\cdot) = 1$ if parameter w seems to be crucial in the current layer, and 0 otherwise

a_k is the pruning threshold. $b_k = a_k + t$, where t is a pre-defined small margin.

Dynamic Network Surgery

1. Choose a neural network architecture.
2. Train the network until a reasonable solution is obtained.
3. Update T_k based on $h_k(\cdot)$.
4. Update W_k based on back-propagation.
5. Iterate to step 3.

Dynamic Network Surgery

model	Top-1 error	Parameters	Iterations	Compression
LeNet-5 reference	0.91%	431K	10K	108×
LeNet-5 pruned	0.91%	4.0K	16K	
LeNet-300-100 reference	2.28%	267K	10K	56×
LeNet-300-100 pruned	1.99%	4.8K	25K	
AlexNet reference	43.42%	61M	450K	17.7×
AlexNet pruned	43.09%	3.45M	700K	

Filter-level Pruning

Motivation: Previous magnitude-based weight pruning produces **irregular sparsity** in the pruned network which requires specialized libraries to achieve acceleration.

Li et al. propose a filter-level pruning algorithm, which identifies less important filters and discards them totally, thus obtaining a **structured** pruned model.

Filter-level Pruning

Method:

1) For each filter $\mathcal{F}_i$, calculate the sum of its absolute kernel weight

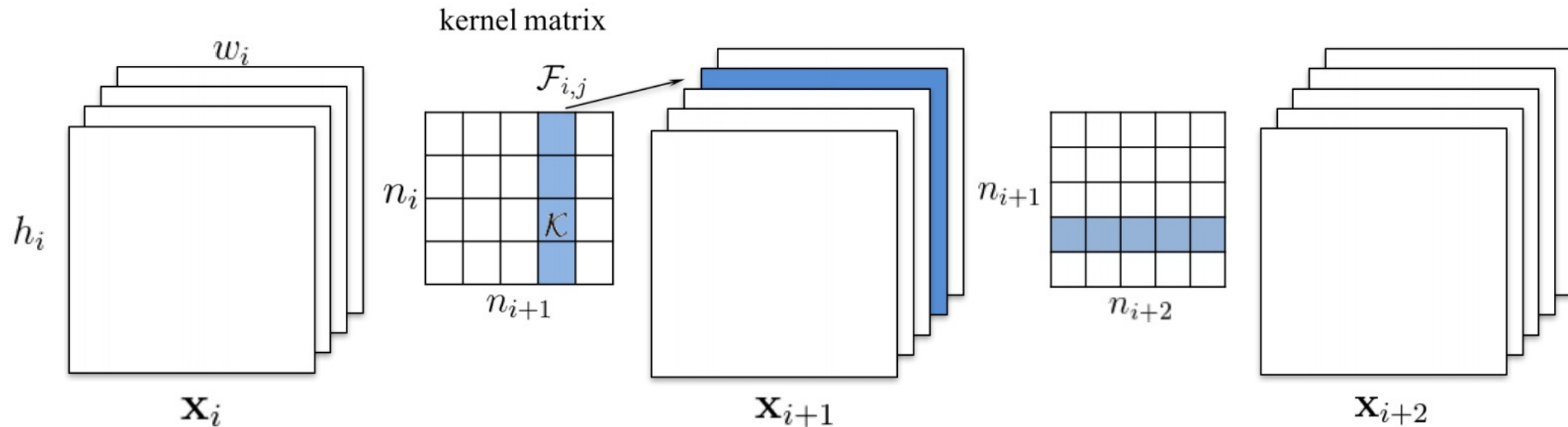
$$s_i = \sum_{j=1}^m \sum_{p=1}^k \sum_{q=1}^k |\mathcal{F}_{i,j,p,q}|$$

2) Sort the filters according to s_i

3) Prune the filter with the smallest value

Here, $\mathcal{F} \in \mathbb{R}^{n \times m \times k \times k}$ is the filters of one layer, k is the kernel size, m, n is the number of input channels and output channels, respectively.

Filter-level Pruning



When a filter $\mathcal{F}_{i,j}$ is pruned, its corresponding feature map $\mathbf{x}_{i+1,j}$ is removed, which reduces $n_i k^2 h_{i+1} w_{i+1}$ operations.

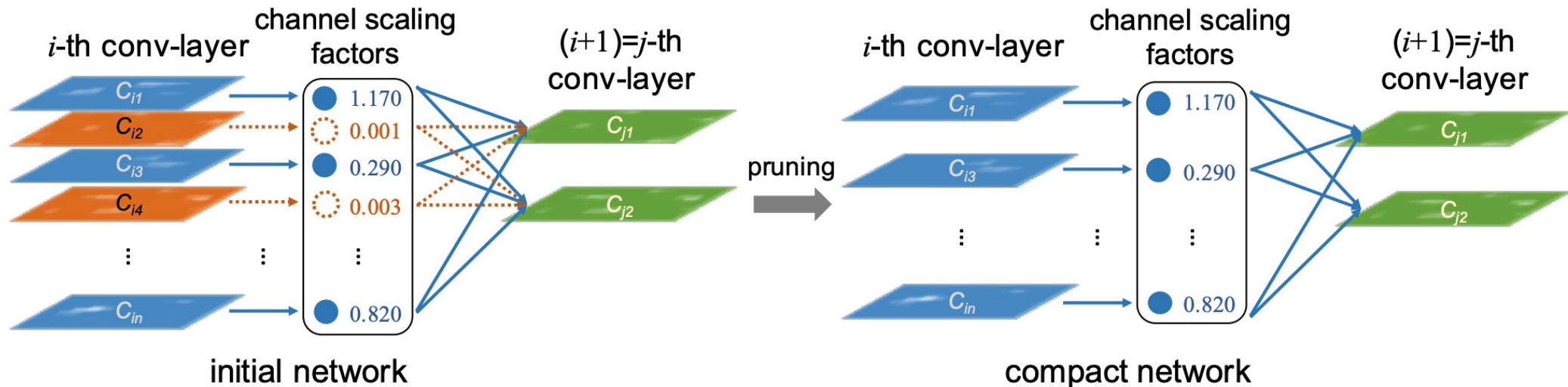
The kernels that apply on the removed feature maps from the filters of the next convolutional layer are also removed, which saves an additional $n_{i+2} k^2 h_{i+2} w_{i+2}$ operations

BN statistics-based Pruning

Main idea:

Leverage the **scaling layers** in batch normalization to effectively identify and prune unimportant channels in the network.

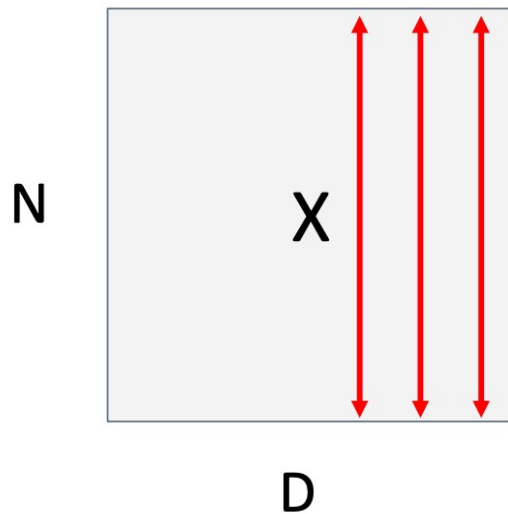
The channels with small scaling factor values (in **orange color**) will be pruned



BN statistics-based Pruning

Remember batch normalization? BN layer normalizes the internal activations using mini-batch statistics

Input: $x : N \times D$



$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$$

Per-channel mean, shape is D

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$$

Per-channel std, shape is D

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

Normalized x,
Shape is N x D

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

Output,
Shape is N x D

BN statistics-based Pruning

Sparsity-induced penalty

- Jointly train the network weights and these scaling factors, with sparsity regularization imposed on the latter

$$L = \sum_{(x,y)} l(f(x, W), y) + \lambda \sum_{\gamma \in \Gamma} g(\gamma)$$


$g(\cdot)$ is a sparsity-induced penalty on the scaling factors
L1-norm is widely used to achieve sparsity

BN statistics-based Pruning

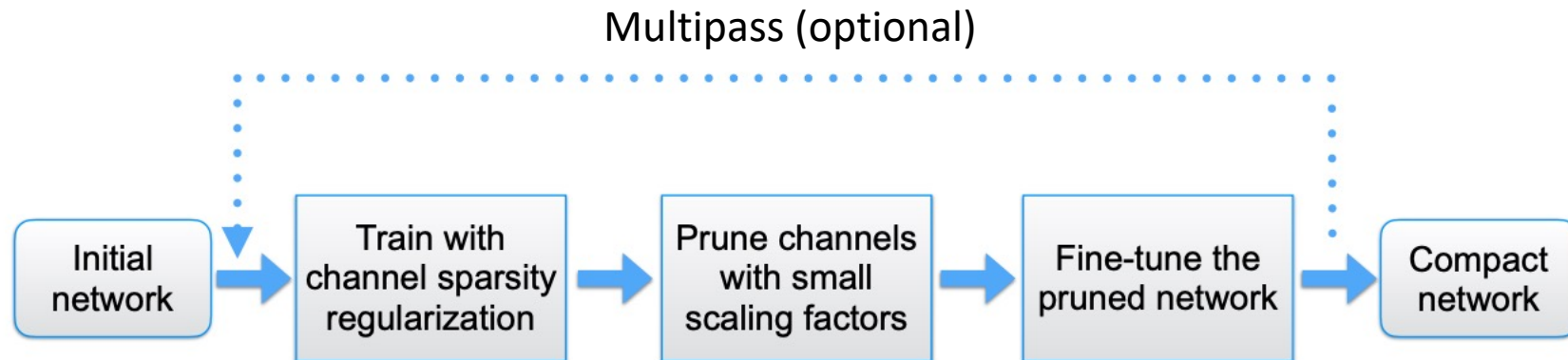
It is common practice to insert a BN layer after a convolutional layer, with channel-wise scaling/shifting parameters.

After training under channel-level **sparsity-induced regularization**, we obtain a model in which many scaling factors γ are near zero

Unimportant channels tend to have small scaling factor γ

BN statistics-based Pruning

1. Choose a neural network architecture.
2. Train with channel sparsity regularization
3. Prune channels with small scaling factors
4. Fine-tune the pruned network
5. Obtain the compact network



Slimmable Neural Networks

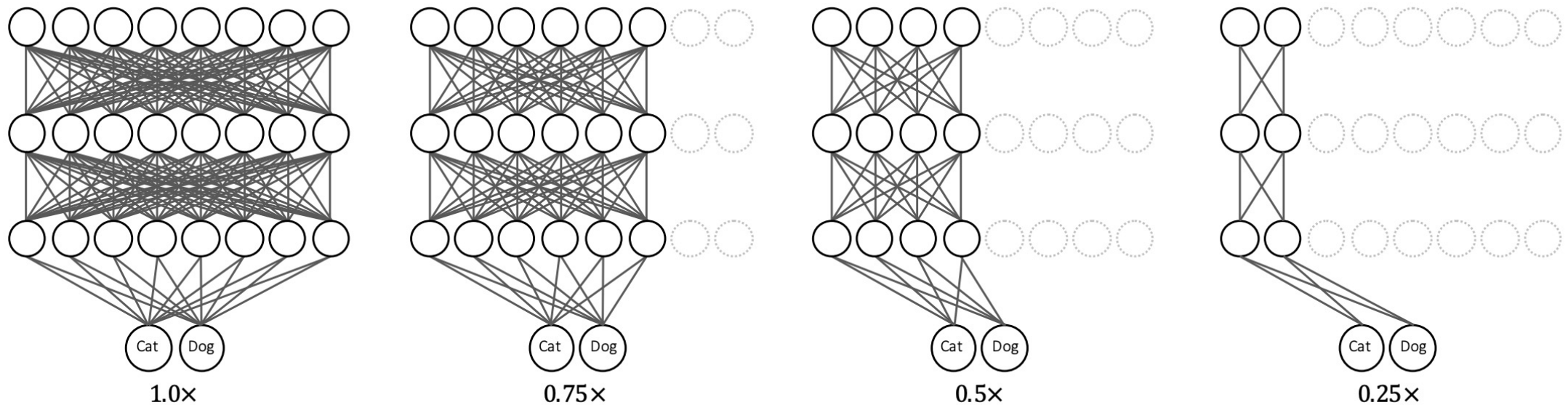
Motivation: Given budgets of resources, how to instantly, adaptively and efficiently trade off between accuracy and latency for neural networks at runtime?

Previous methods requires the training of multiple networks and are very costly.

Slimmable Neural Networks

The trained SuperNet can produce a collection of sub-structures whose

- performance is on par with their train-from-scratch counterparts
- satisfy different FLOPs requirement and do not need retraining



Slimmable Neural Networks

Switchable Batch Normalization employs independent BN for different switches in a slimmable network

Algorithm 1 Training slimmable neural network M .

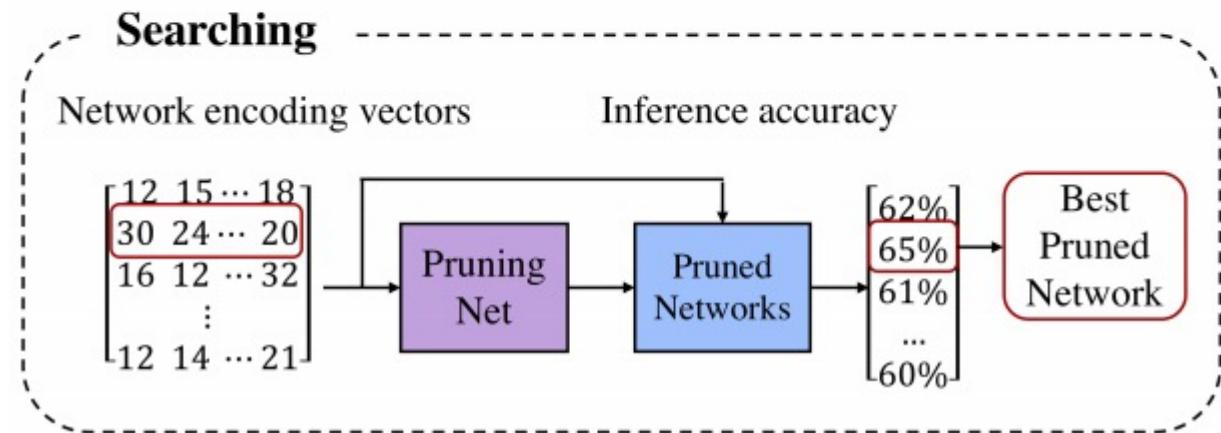
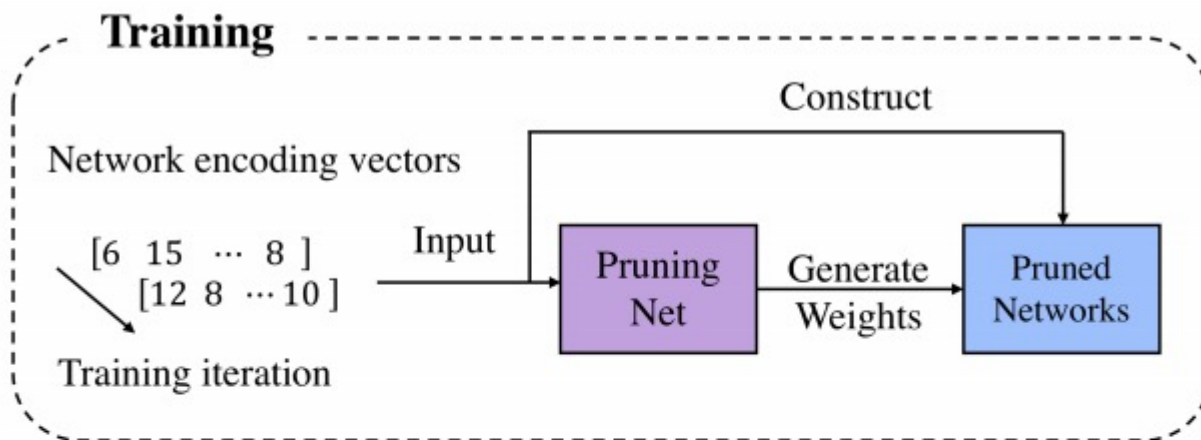
Require: Define *switchable width list* for slimmable network M , for example, $[0.25, 0.5, 0.75, 1.0] \times$.

```
1: Initialize shared convolutions and fully-connected layers for slimmable network  $M$ .
2: Initialize independent batch normalization parameters for each width in switchable width list.
3: for  $i = 1, \dots, n_{iters}$  do
4:   Get next mini-batch of data  $x$  and label  $y$ .
5:   Clear gradients of weights,  $optimizer.zero\_grad()$ .
6:   for width in switchable width list do
7:     Switch the batch normalization parameters of current width on network  $M$ .
8:     Execute sub-network at current width,  $\hat{y} = M'(x)$ .
9:     Compute loss,  $loss = criterion(\hat{y}, y)$ .
10:    Compute gradients,  $loss.backward()$ .
11:  end for
12:  Update weights,  $optimizer.step()$ .
13: end for
```

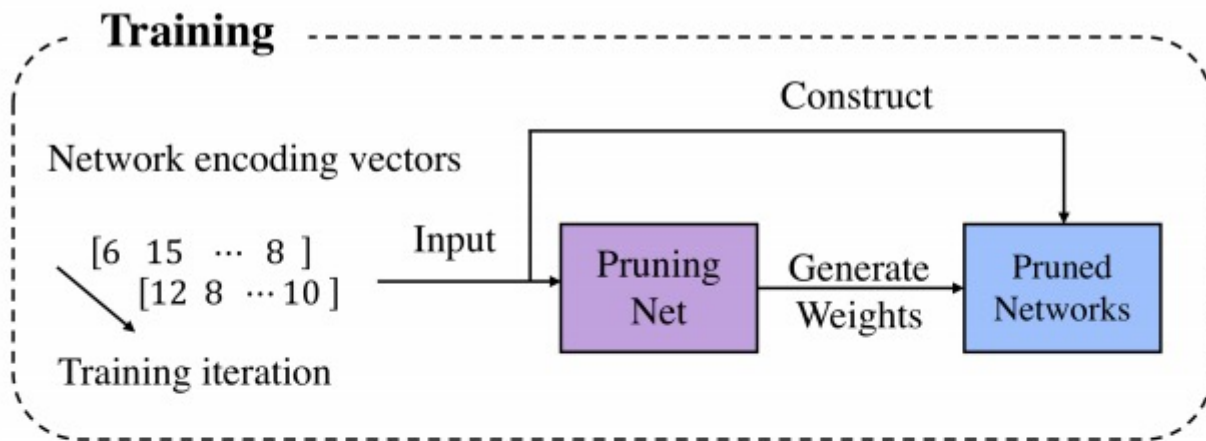
accumulate back-propagated gradients of all switches, and update weights afterwards

Meta Pruning

- Two steps
 - **Training a PruningNet** - generate weight parameters for any pruned structure given the target network
 - **Searching for the best Pruned Network** - search good-performing structures by just evaluating their accuracy on the validation data

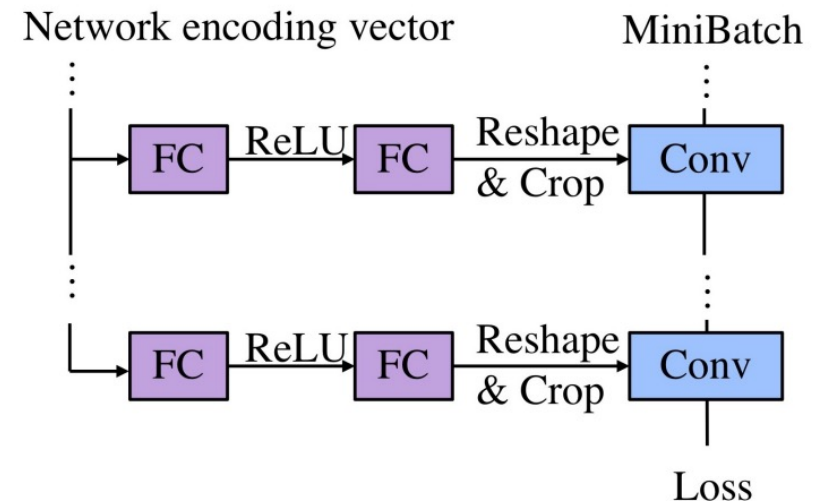


Meta Pruning



1. Training step

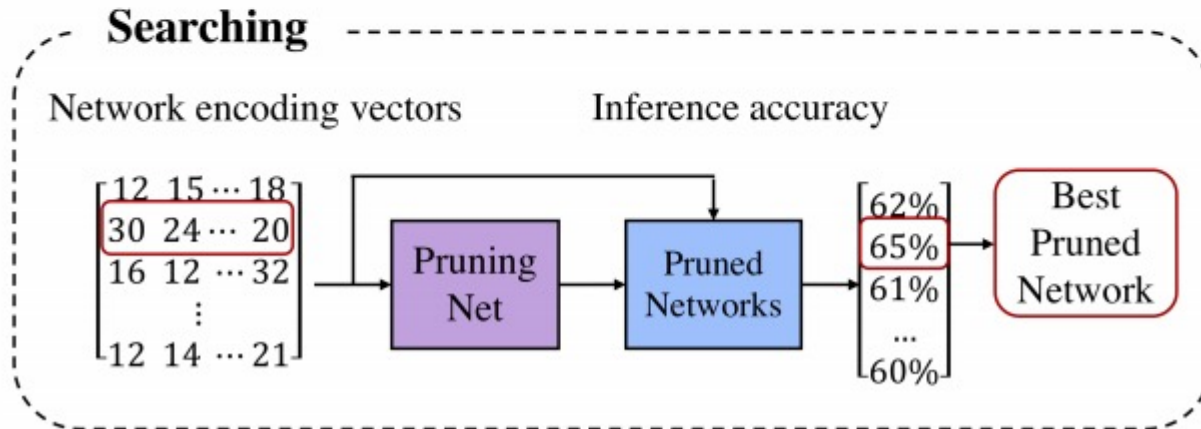
- At each iteration, a network encoding vector (i.e., the number of channels in each layer) is randomly generated.
- The PruningNet takes the network encoding vector and generate the weights for the Pruned Network.



The PruningNet is composed of a series of PruningNet blocks and each block is comprised of two fully connected layers.

The produced weight is reshaped and cropped to match the number of input and output channels of the pruned network

Meta Pruning



2. Searching for the best Pruned Network

- Construct many Pruned Networks by varying network encoding vector and evaluate their performance on the validation set with the weights predicted by the PruningNet
- No finetuning or re-training is needed at search time.

Algorithm 1 Evolutionary Search Algorithm

Hyper Parameters: Population Size: $\mathcal{P}$, Number of Mutation: $\mathcal{M}$, Number of Crossover: $\mathcal{S}$, Max Number of Iterations: $\mathcal{N}$.

Input: PruningNet: *PruningNet*, Constraints: $\mathcal{C}$.

Output: Most accurate gene: $\mathcal{G}_{top}$.

```

1:  $\mathcal{G}_0 = \text{Random}(\mathcal{P})$ , s.t.  $\mathcal{C}$ ;
2:  $\mathcal{G}_{topK} = \emptyset$ ;
3: for  $i = 0 : \mathcal{N}$  do
4:    $\{\mathcal{G}_i, \text{accuracy}\} = \text{Inference}(\text{PruningNet}(\mathcal{G}_i))$ ;
5:    $\mathcal{G}_{topK}, \text{accuracy}_{topK} = \text{TopK}(\{\mathcal{G}_i, \text{accuracy}\})$ ;
6:    $\mathcal{G}_{mutation} = \text{Mutation}(\mathcal{G}_{topK}, \mathcal{M})$ , s.t.  $\mathcal{C}$ ;
7:    $\mathcal{G}_{crossover} = \text{Crossover}(\mathcal{G}_{topK}, \mathcal{S})$ , s.t.  $\mathcal{C}$ ;
8:    $\mathcal{G}_i = \mathcal{G}_{mutation} + \mathcal{G}_{crossover}$ ;
9: end for
10:  $\mathcal{G}_{top1}, \text{accuracy}_{top1} = \text{Top1}(\{\mathcal{G}_{\mathcal{N}}, \text{accuracy}\})$ ;
11: return  $\mathcal{G}_{top1}$ ;

```

Mutation: changing a proportion of elements in the gene randomly

Crossover: randomly recombine the genes in two parent genes to generate an off-spring

Differentiable Markov Channel Pruning

Guo et al. model channel pruning as a **Markov process**, where the state S_k denotes the k-th channel is kept in a layer and the transition from S_k to S_{k+1} represents the probability of retaining the (k+1)-th channel given that the k-th channel is kept.

Let $p(w_1, \dots, w_{k-1})$ be the probability that first (k-1) channels are retained,
Then, the probability of retaining first k channels can be represented as

$$p(w_1, \dots, w_k) = p(w_k | w_1, \dots, w_{k-1}) p(w_1, \dots, w_{k-1})$$

conditional probability

Since retaining w_k is conditionally independent of $\{w_1, \dots, w_{k-2}\}$ given w_{k-1} is retained, then

$$p_k = p(w_k | w_1, \dots, w_{k-1}) = p(w_k | w_{k-1})$$

Differentiable Markov Channel Pruning

Here, we leave at least one channel for each layer, so $p(w_1) = 1$

The marginal probability of sampling channel w_k is

$$p(w_k) = p(w_k|w_{k-1})p(w_{k-1}) + p(w_k|\neg w_{k-1})p(\neg w_{k-1})$$

$$= p(w_k|w_{k-1})p(w_{k-1}) + 0$$

$$= p(w_1) \prod_{i=2}^k p(w_i|w_{i-1})$$

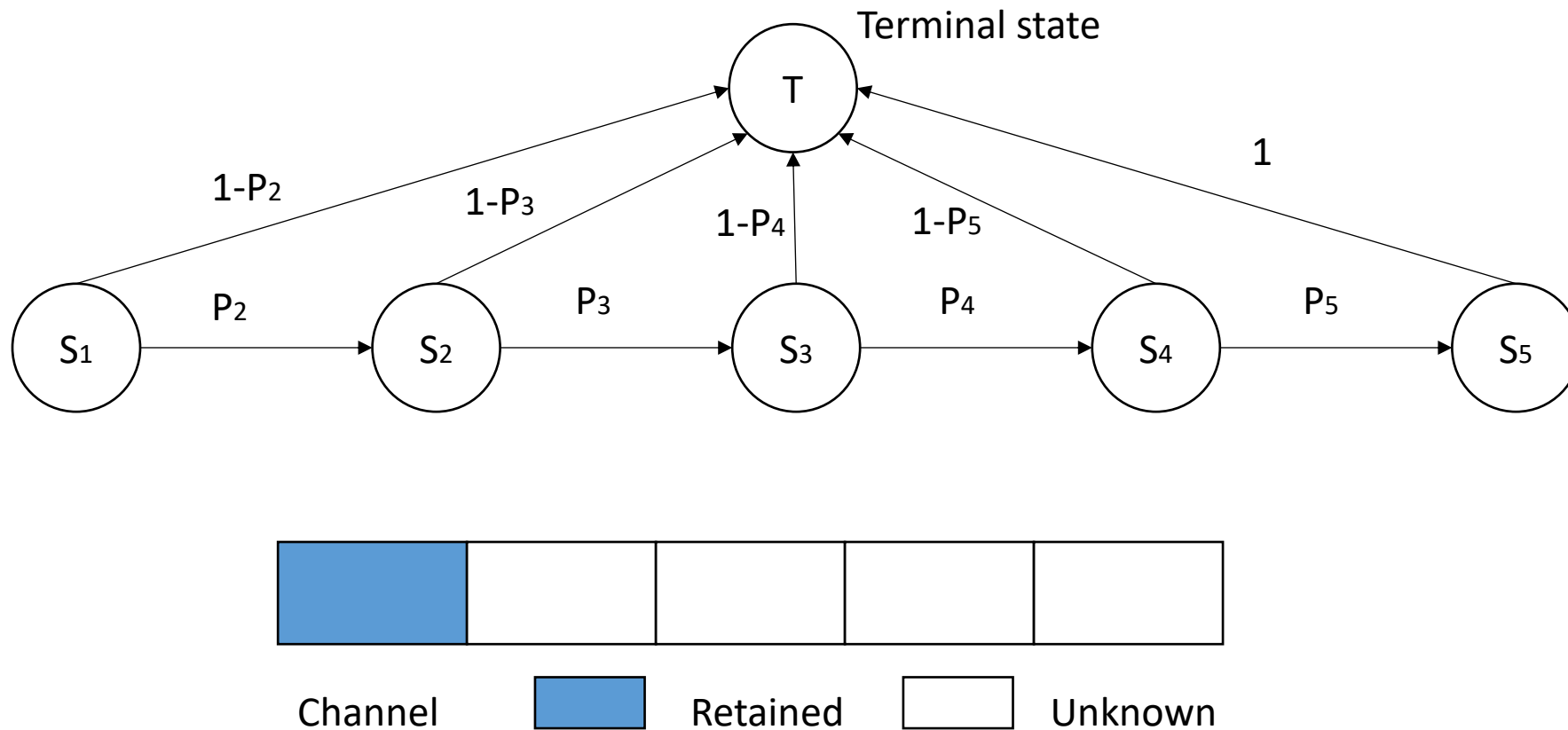
$$= \prod_{i=1}^k p_i$$

$\neg w_{k-1}$

means (k-1)-th channel
is discarded

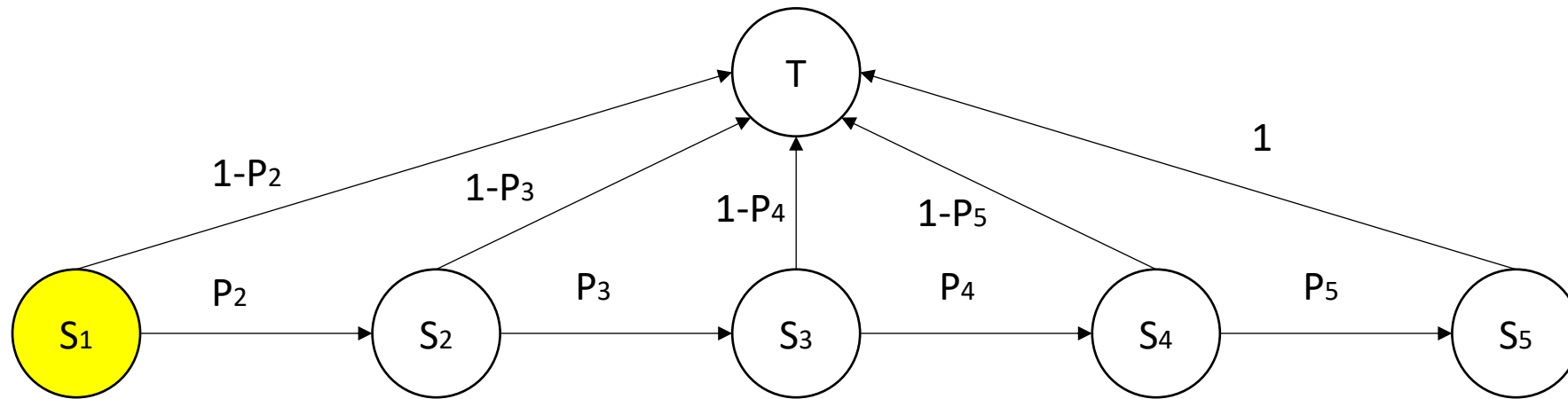
Differentiable Markov Channel Pruning

Example: Suppose we need to prune a layer which has five channels,



Differentiable Markov Channel Pruning

The first channel is obviously kept



Channel



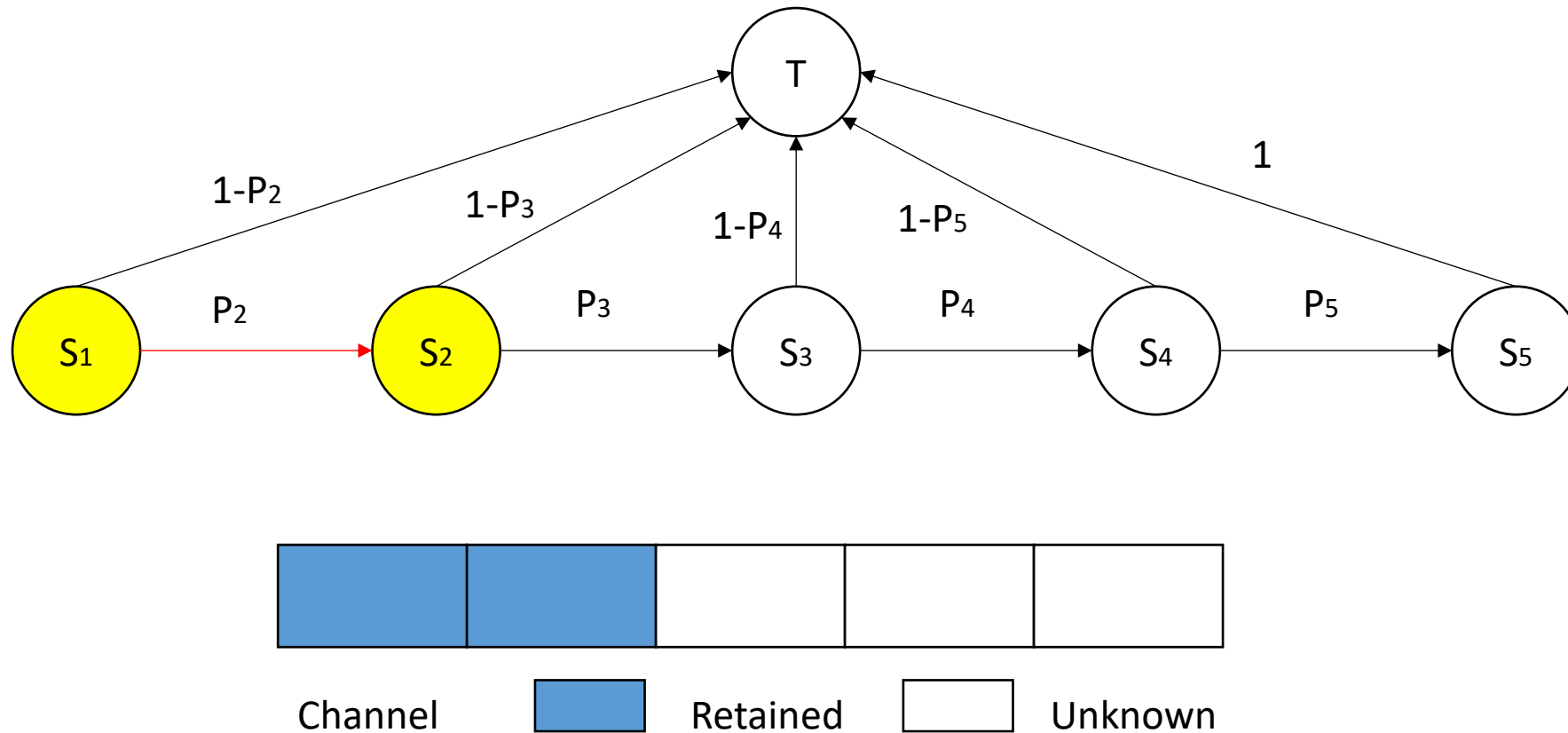
Retained



Unknown

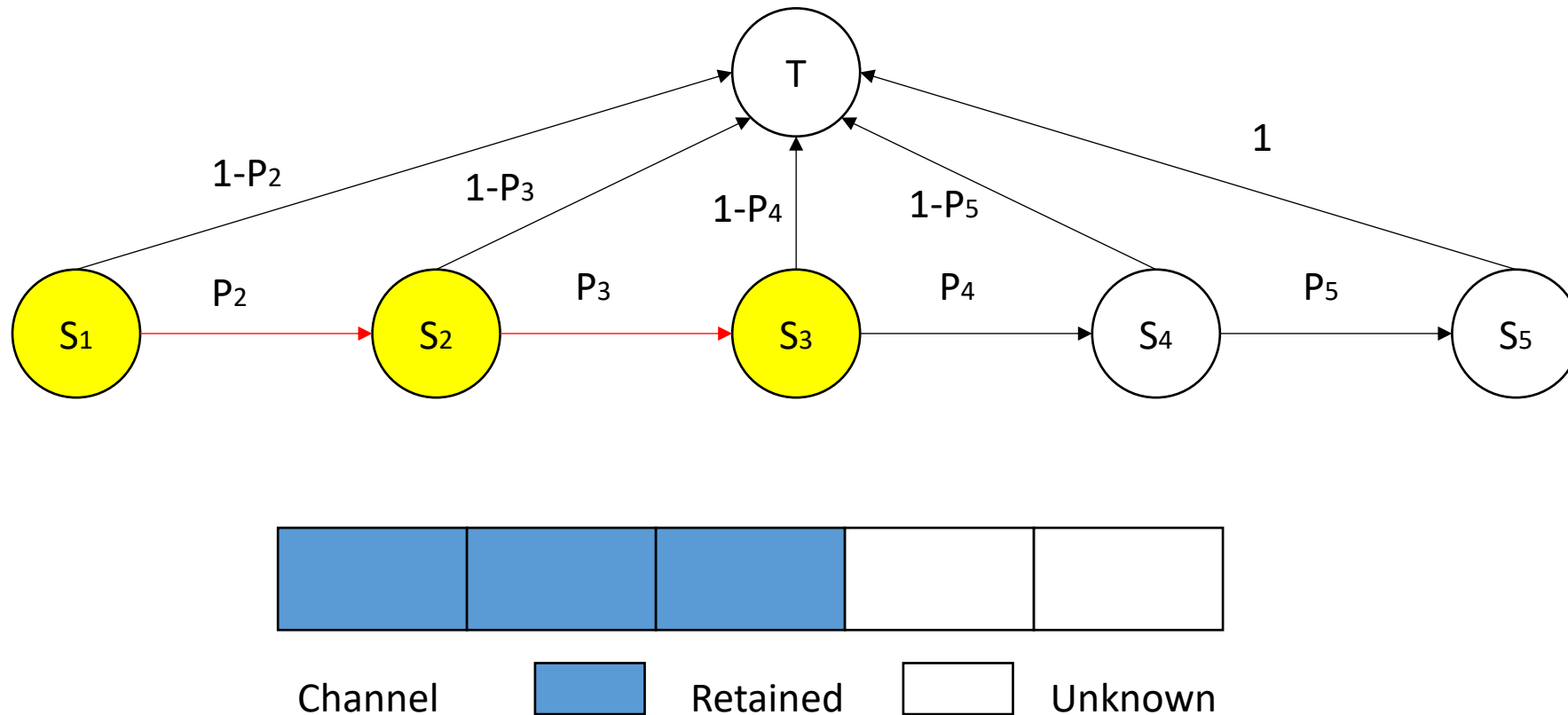
Differentiable Markov Channel Pruning

Then we sample the second channel and it is kept with probability p_2



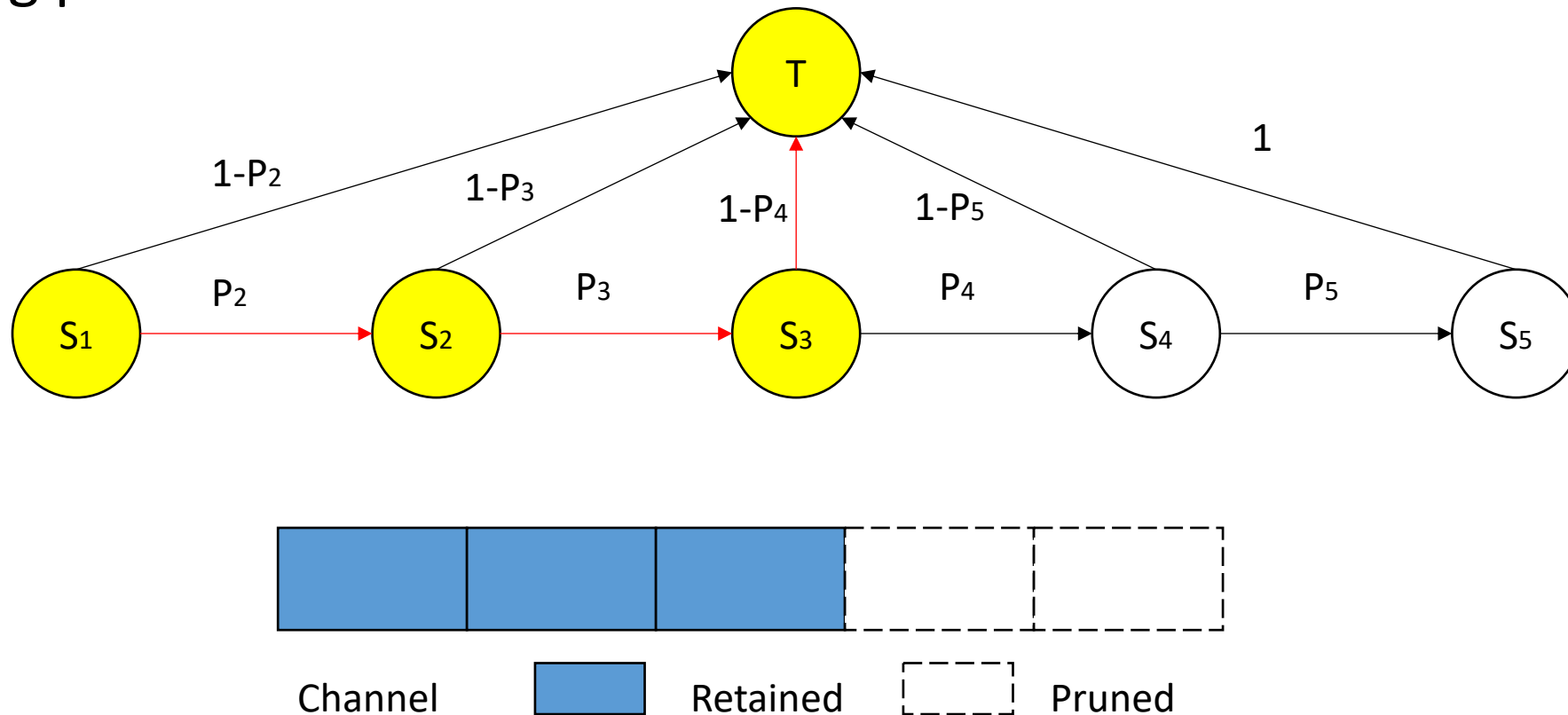
Differentiable Markov Channel Pruning

We sample the third channel and it is kept with probability p_3



Differentiable Markov Channel Pruning

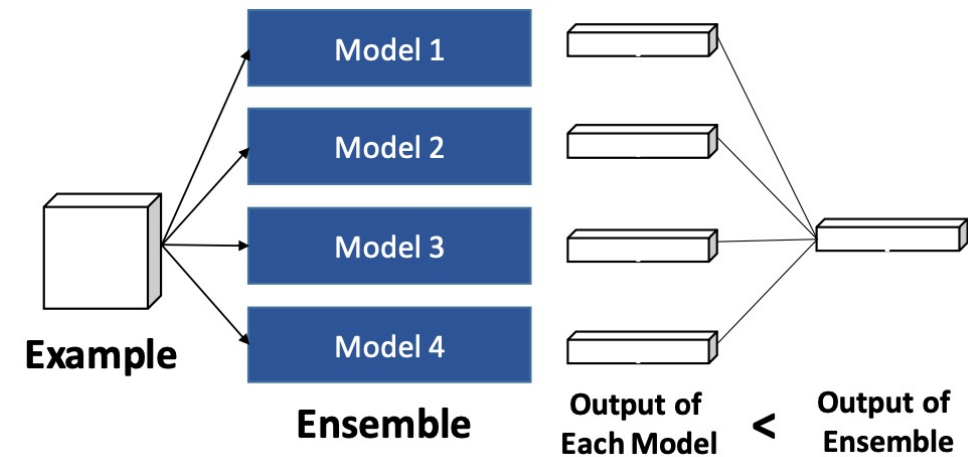
we sample the fourth channel and it is discarded with probability $(1-p_4)$; the sampling process terminates



Knowledge Distillation

The conflicting constraints of learning and using

- The easiest way to extract a lot of knowledge from the training data is to learn many different models in parallel.
 - We want to make the models as different as possible to **minimize the correlations** between their errors.
 - We can use **different initializations** or **different architectures** or different subsets of the training data.
 - It is helpful to **over-fit** the individual models.
- At test time we **average the predictions** of all the models or of a selected subset of good models that make different errors.
 - That's how almost all ML competitions are won



Why ensembles are bad at test time

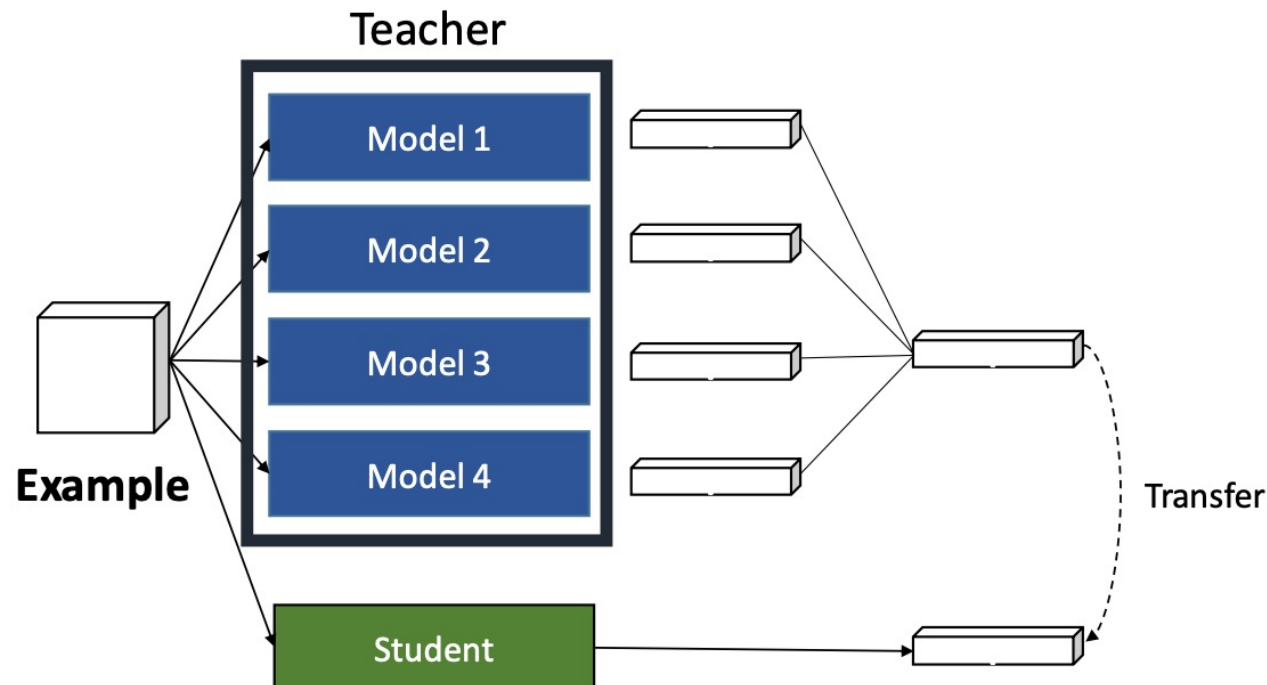
- A big ensemble is highly redundant. It has very very little knowledge **per parameter**.
- At test time we want to minimize the amount of computation and the memory footprint. These constraints are generally much more severe at test time than during training.

Knowledge Distillation

- The ensemble implements a function from input to output.
- Forget the models in the ensemble and the way they are parameterized and focus on the function
 - After learning the ensemble, we have our hands on the function.
 - Can we transfer the knowledge in the function into a single smaller model?

Knowledge Distillation

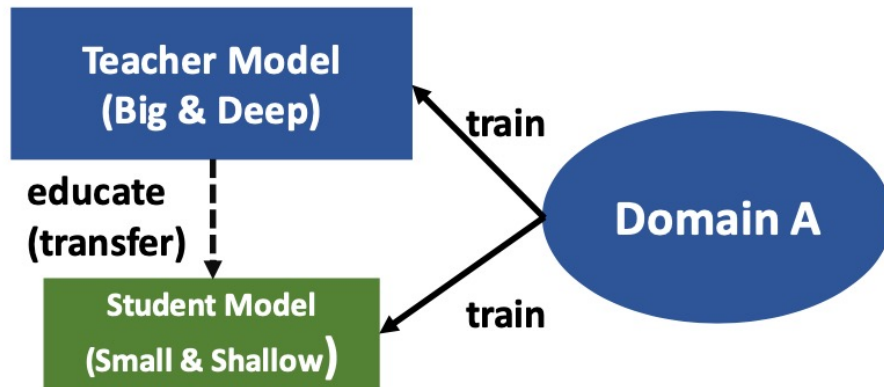
- General idea
 - Try to approximate bigger/more complex neural nets (models) with smaller neural nets (models) of similar generalization performance.
 - Have a student model model the teacher's function *directly*.



Knowledge Distillation

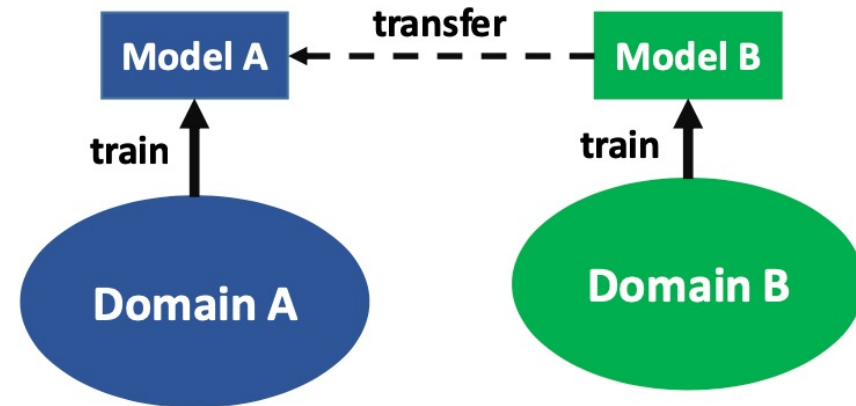
Differences between knowledge distillation and transfer learning

Knowledge Distillation (Transfer)



- For model compression
- To improve performance of student over teacher

Transfer Learning



- When data is not sufficient.
- When label for a problem is not presented.
- E.g., pretrained-model on ImageNet

Knowledge Distillation

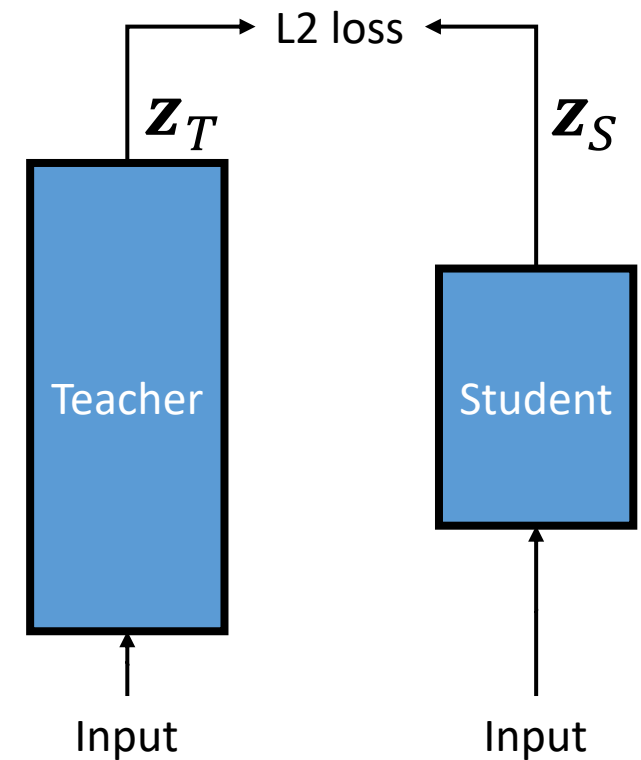
How to transfer knowledge? Remember the softmax?

$$\mathbf{z} \begin{bmatrix} 2.0 \\ 1.0 \\ 0.1 \end{bmatrix} \begin{matrix} \longrightarrow \\ \longrightarrow \\ \longrightarrow \end{matrix} \boxed{S(z_i) = \frac{e^{z_j}}{\sum_j e^{z_j}}} \begin{matrix} \longrightarrow \\ \longrightarrow \\ \longrightarrow \end{matrix} \begin{matrix} p = 0.66 \\ p = 0.24 \\ p = 0.10 \end{matrix}$$

Numeric output of the last linear layer of a multi-class classification neural network

Transfer logits

- Model compression
 - Try to match the **logits** (the input to the softmax layer) produced by the teacher net
 - Treated as a regression problem (regressing the logit with L2-loss)
- Steps
 - Feed teacher with data
 - Obtain logits from teacher (transfer training set)
 - Train student on these logits (regression)



Unlabeled data can now be used to train the student!

Transfer logits

- Issues: If the output is a big N-way softmax, the targets are usually a single 1 and a whole lot of 0's.
- How to reveal more information about the function on each training case?

Transfer soft targets

- Dark knowledge
 - Try to match **soft targets** produced by the teacher net
- Softmax layer with **temperature, τ** (normally set to 1)

$$S(z_i) = \frac{e^{z_j/\tau}}{\sum_j e^{z_j/\tau}}$$

- This will **soften** the posterior distribution
- Using a higher value for **τ** produces a softer posterior distribution over classes

Transfer soft targets

- **Why does it work?** Most of the knowledge in the learned model is in the **relative probabilities** of extremely improbable wrong answers

- Example:

- Truth:

$$\mathbf{y} = \underbrace{(0 \quad 1 \quad 0 \quad 0)}_{(\mathbb{P}_{\text{cow}} \quad \mathbb{P}_{\text{dog}} \quad \mathbb{P}_{\text{cat}} \quad \mathbb{P}_{\text{boat}})}^\top$$

- Teacher output:

$$\hat{\mathbf{y}} = (10^{-6} \quad 0.9 \quad 0.1 \quad 10^{-9})^\top$$

- Softened output:

$$\tilde{\mathbf{y}} = (0.05 \quad 0.4 \quad 0.3 \quad 10^{-3})^\top$$

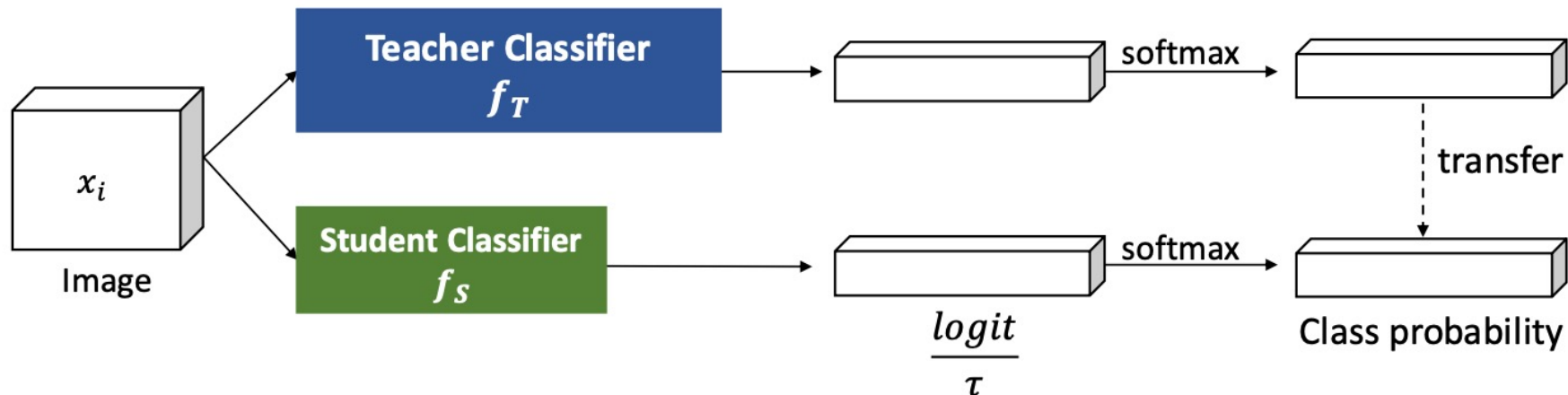
Softened outputs
reveal the dark
knowledge!

Transfer soft targets

- Steps

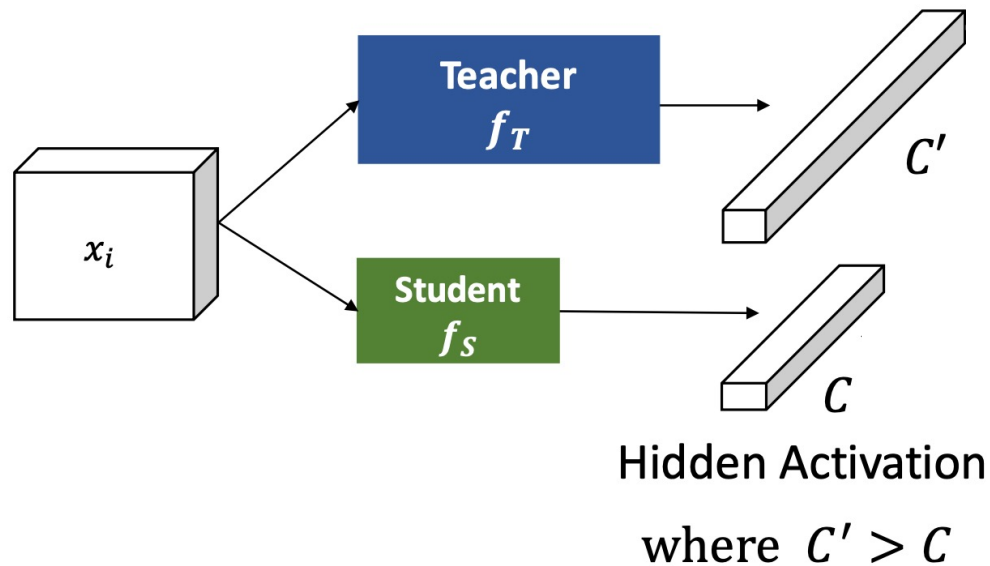
- Feed teacher with data
- Obtain soft targets from teacher (transfer training set)
- Train student on soft targets (with KL divergence) - works better to fit both the hard targets and the soft targets

Objective:
$$\sum_{x_i \in \mathcal{X}} \text{KL} \left(\text{softmax} \left(\frac{f_T(x_i)}{\tau} \right), \text{softmax} \left(\frac{f_S(x_i)}{\tau} \right) \right)$$



Transfer hidden activation

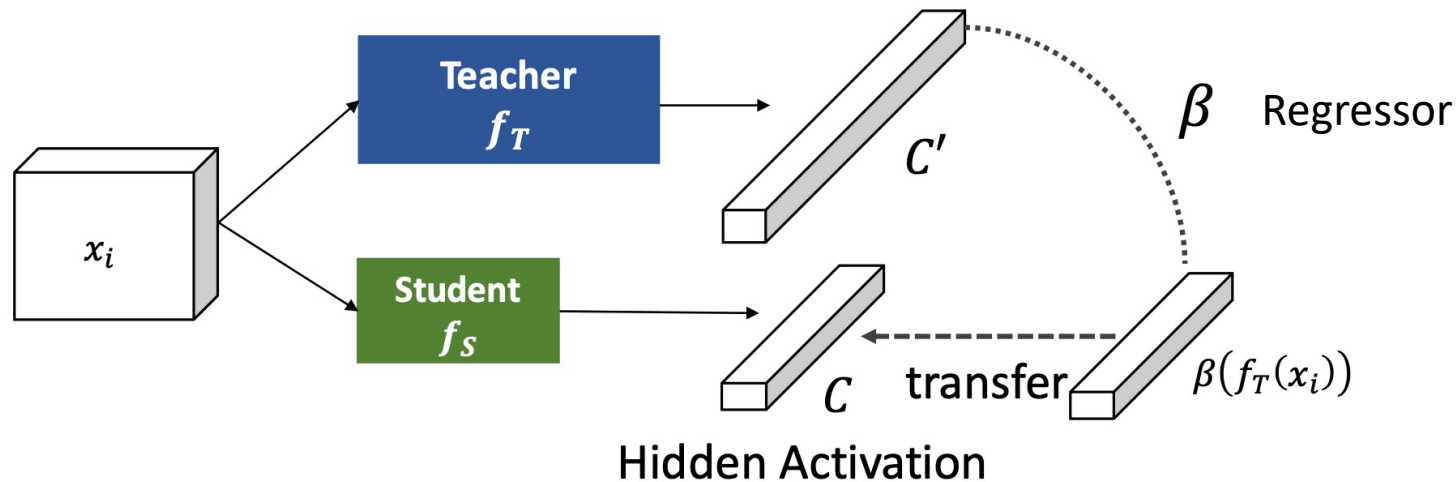
- Use **intermediate representations** learned by the teacher as hints to improve the training process and final performance of the student
- What happen if a student is deeper and thinner than the teacher?
 - Map the student hidden layer to the prediction of the teacher hidden layer



Transfer hidden activation

- Add a regressor to the guided layer, whose output matches the size of the hint layer
- Train the student from the first layer up to the guided layer as well as the regressor parameters

- Minimizing Objective:
$$\sum_{x_i \in \mathcal{X}} \|f_T(x_i) - \beta(f_S(x_i))\|_2^2$$



where $C' > C$

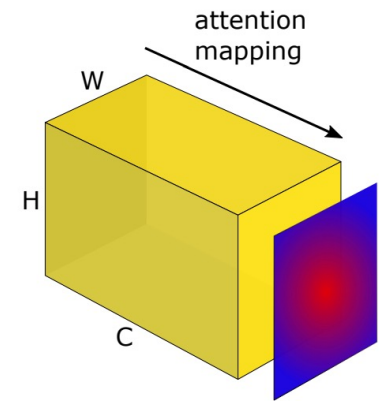
Transfer attention

- Attention plays a critical role in human visual experience
- In CNN, attention is a set of spatial maps that encodes on which spatial areas of the input the network focuses most for taking its output decision
- Activation-based attention

- sum of absolute values: $F_{\text{sum}}(A) = \sum_{i=1}^C |A_i|$

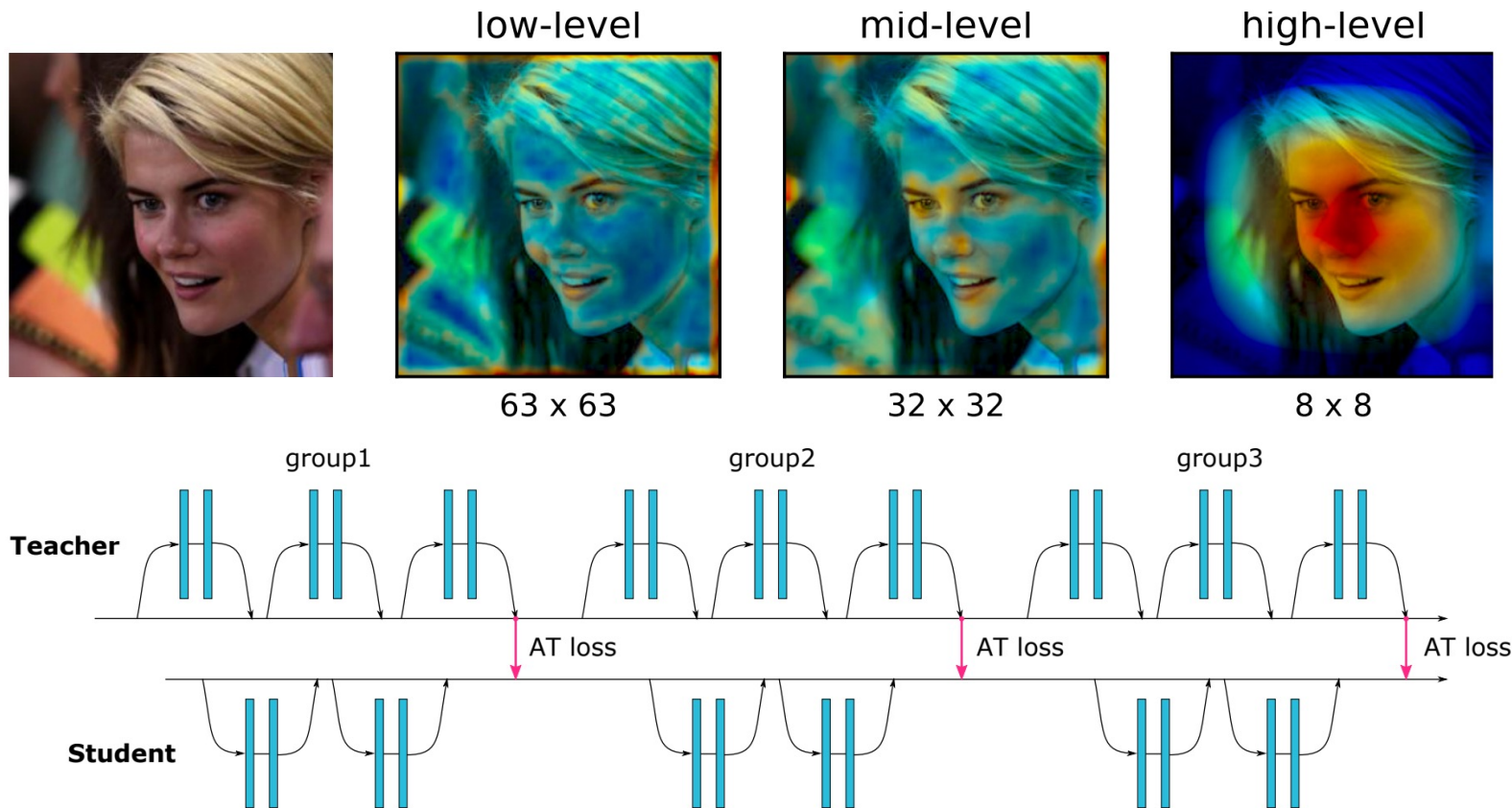
- sum of absolute values raised to the power of p (where $p > 1$): $F_{\text{sum}}^p(A) = \sum_{i=1}^C |A_i|^p$

- max of absolute values raised to the power of p (where $p > 1$): $F_{\text{max}}^p(A) = \max_{i=1, C} |A_i|^p$



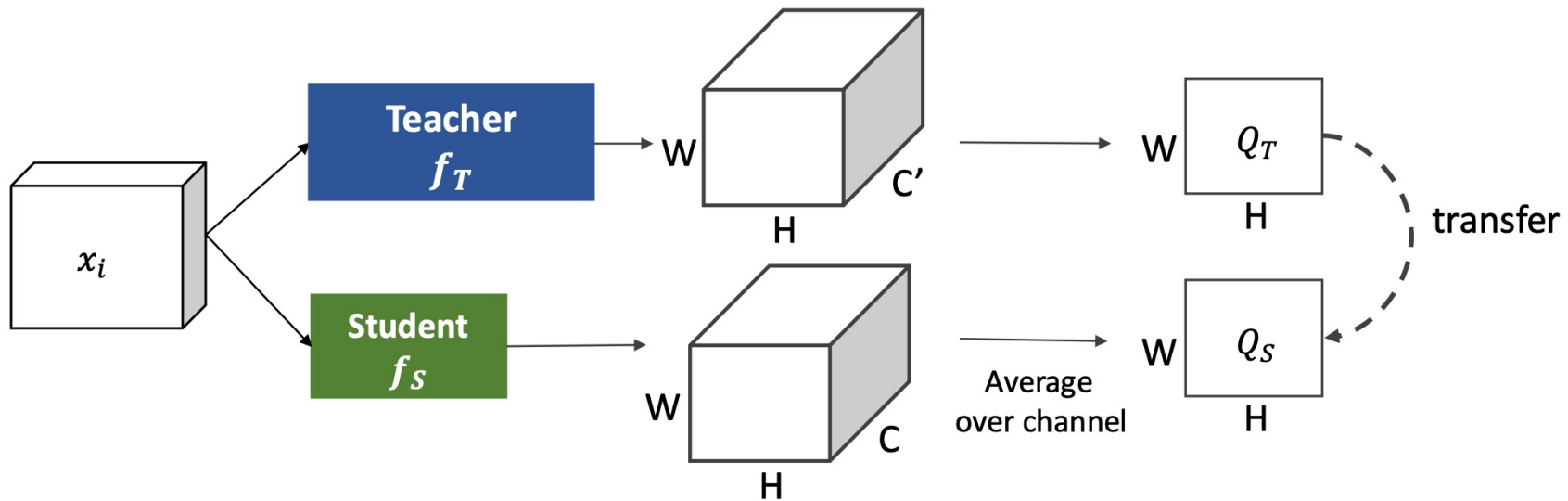
Transfer attention

- Attention maps focus on different parts for different layers in the network



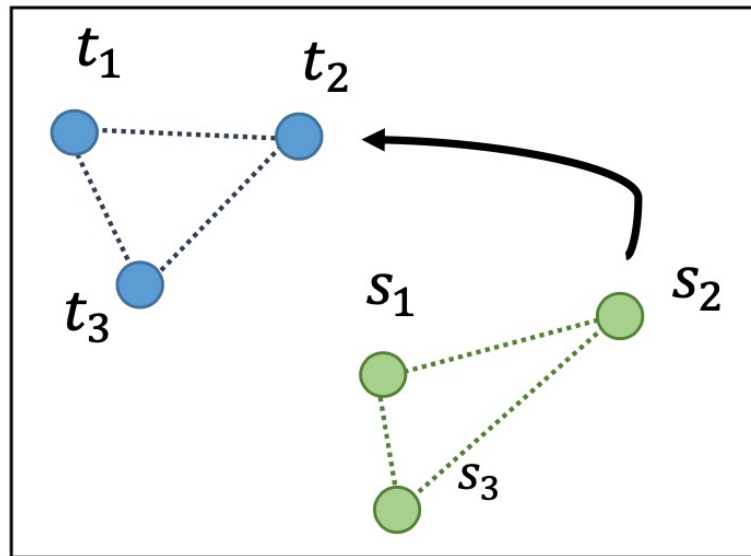
Transfer attention

Minimizing Objective:
$$\sum_{x_i \in \mathcal{X}} \left\| \frac{Q_T^i}{\|Q_T^i\|} - \frac{Q_S^i}{\|Q_S^i\|} \right\|_2$$



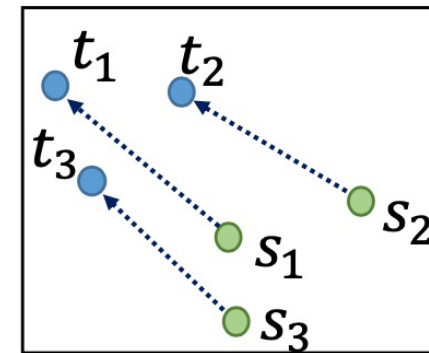
Transfer relation

- Transfers relation among examples represented by teacher to student



Structure to Structure
Relational KD

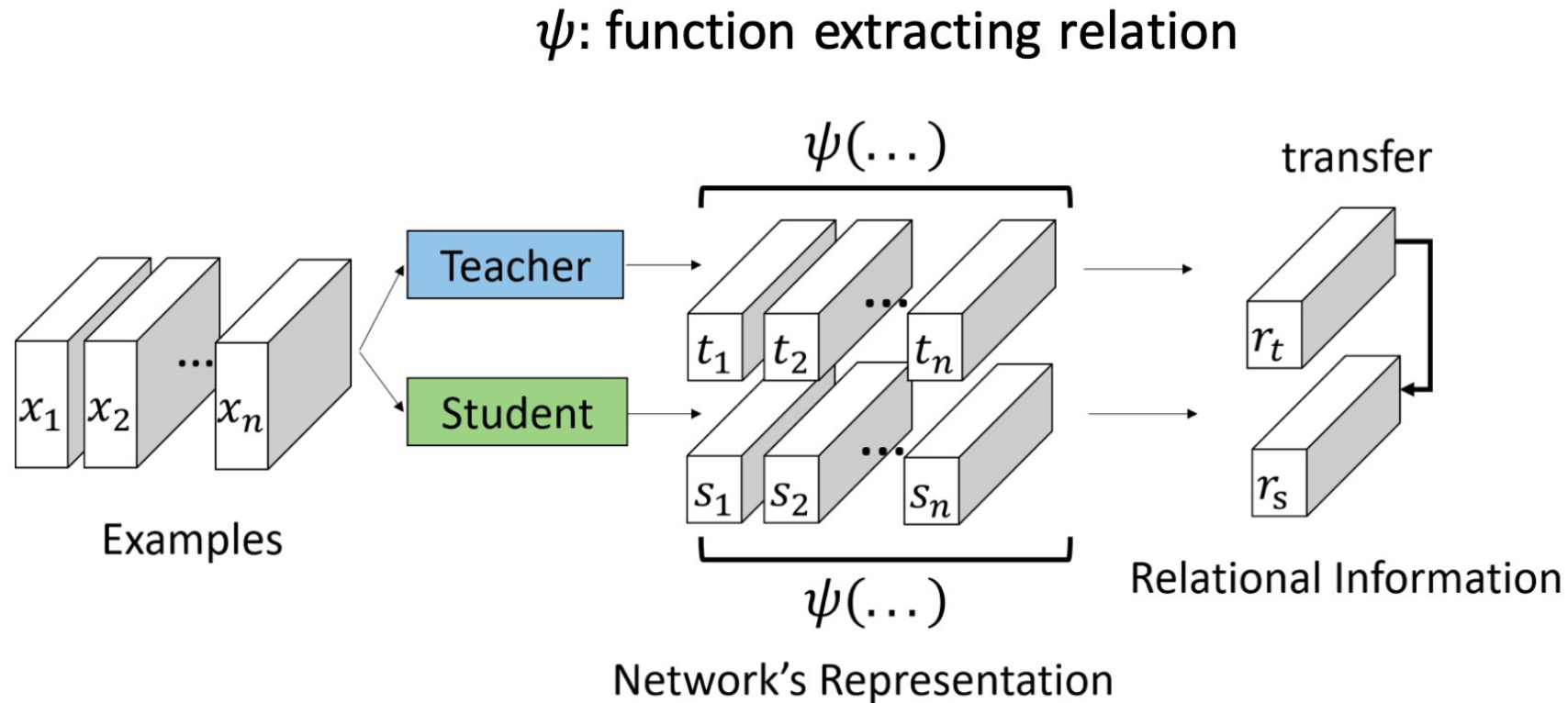
vs.



Point to Point
Individual KD

Transfer relation

- Transfers relation among examples represented by teacher to student



Transfer relation

- Distance-wise loss

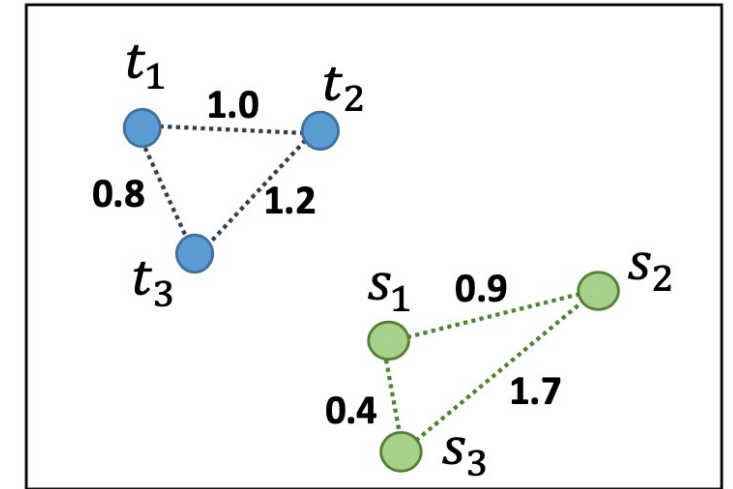
- Transfers relative distance between points on embedding space

$$\psi_D(t_i, t_j) = \frac{1}{\mu_t} \|t_i - t_j\|_2$$
$$\mu_t = \frac{1}{|\mathcal{X}^2|} \sum_{(x_i, x_j) \in \mathcal{X}^2} \|t_i - t_j\|_2$$

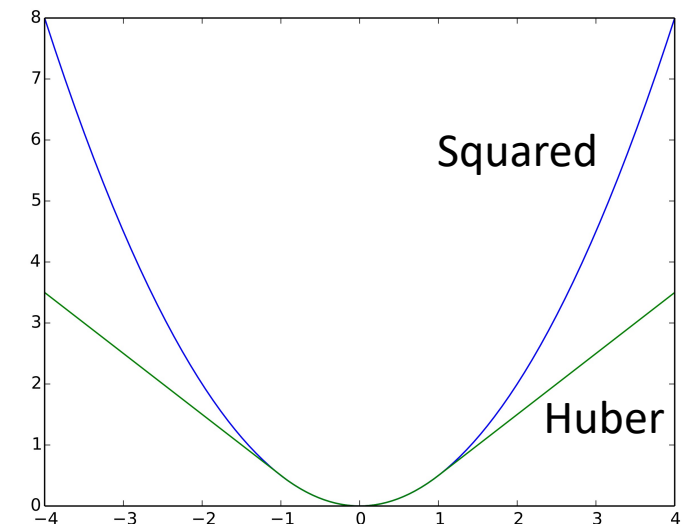
$$\mathcal{L}_{\text{RKD-D}} = \sum_{(x_i, x_j) \in \mathcal{X}^2} l_\delta(\psi_D(t_i, t_j), \psi_D(s_i, s_j))$$

Where l_δ is Huber loss:

$$l_\delta(x, y) = \begin{cases} \frac{1}{2}(x - y)^2 & \text{for } |x - y| \leq 1, \\ |x - y| - \frac{1}{2}, & \text{otherwise.} \end{cases}$$



Embedding Space



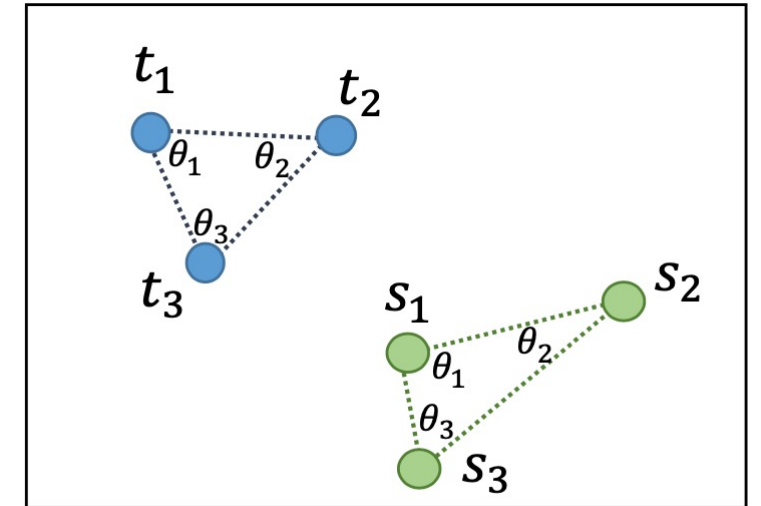
Transfer relation

- Angle-wise loss
 - Transfers angle formed by three points on embedding space

$$\psi_A(t_i, t_j, t_k) = \cos \angle t_i t_j t_k = \langle \mathbf{e}^{ij}, \mathbf{e}^{kj} \rangle$$

$$\text{where } \mathbf{e}^{ij} = \frac{t_i - t_j}{\|t_i - t_j\|_2}, \mathbf{e}^{kj} = \frac{t_k - t_j}{\|t_k - t_j\|_2}$$

$$\mathcal{L}_{\text{RKD-A}} = \sum_{(x_i, x_j, x_k) \in \mathcal{X}^3} l_\delta(\psi_A(t_i, t_j, t_k), \psi_A(s_i, s_j, s_k))$$



Embedding Space

Transfer relation

- Where to apply
 - On any hidden layers or embedding layers.
 - Not on layer where individual output values are crucial.
Because, RKD does not transfer output value of individual examples. E.g., softmax layer for classification.

Application – transfer hidden activation

- Steering angle prediction given an image/video clip
- At the same time learns a small network

- Udacity

[Udacity 2018]



Challenges:

- Curved roads
- Challenging light conditions

- Comma.ai

[Santana and Hotz 2016]



Challenges:

- Curved roads
- Busy traffic

- BDD100K

[Yu et al. 2018]

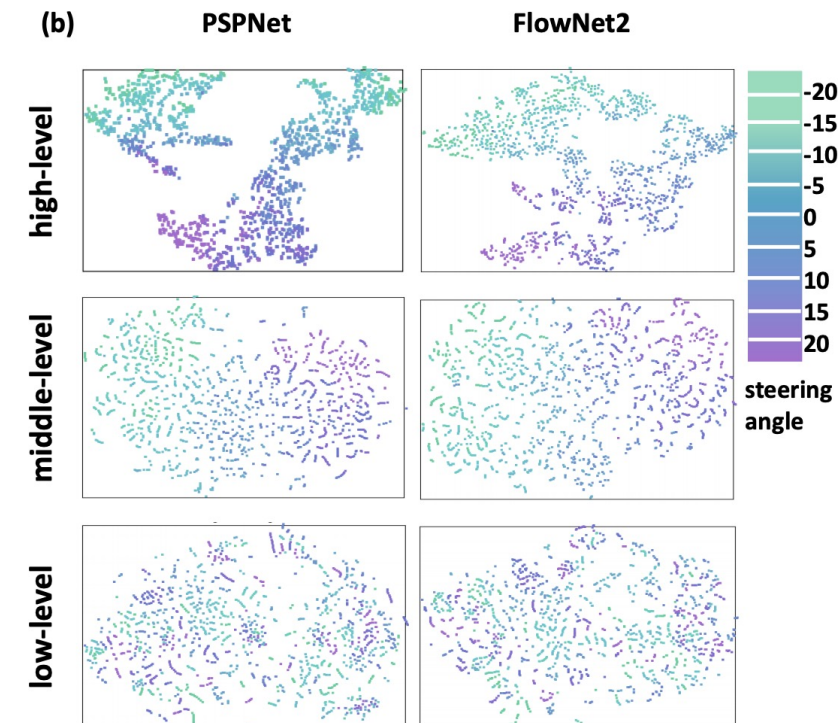
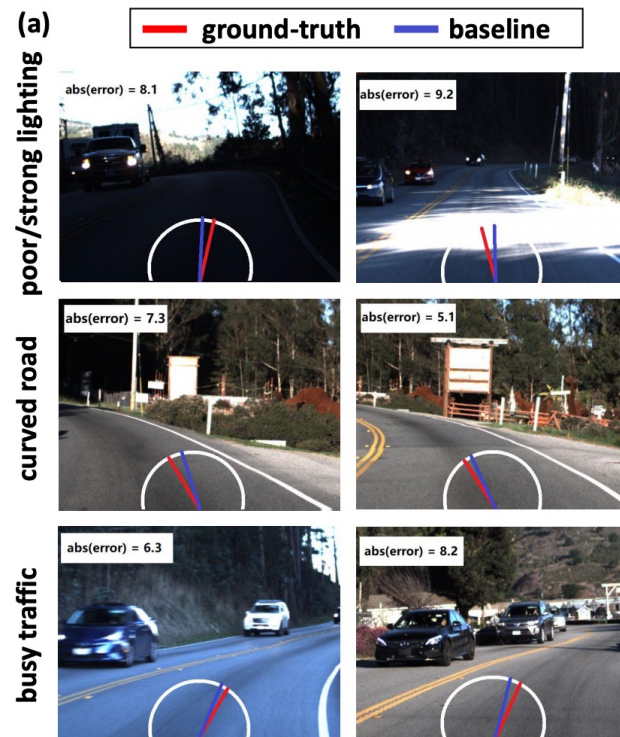


Challenges:

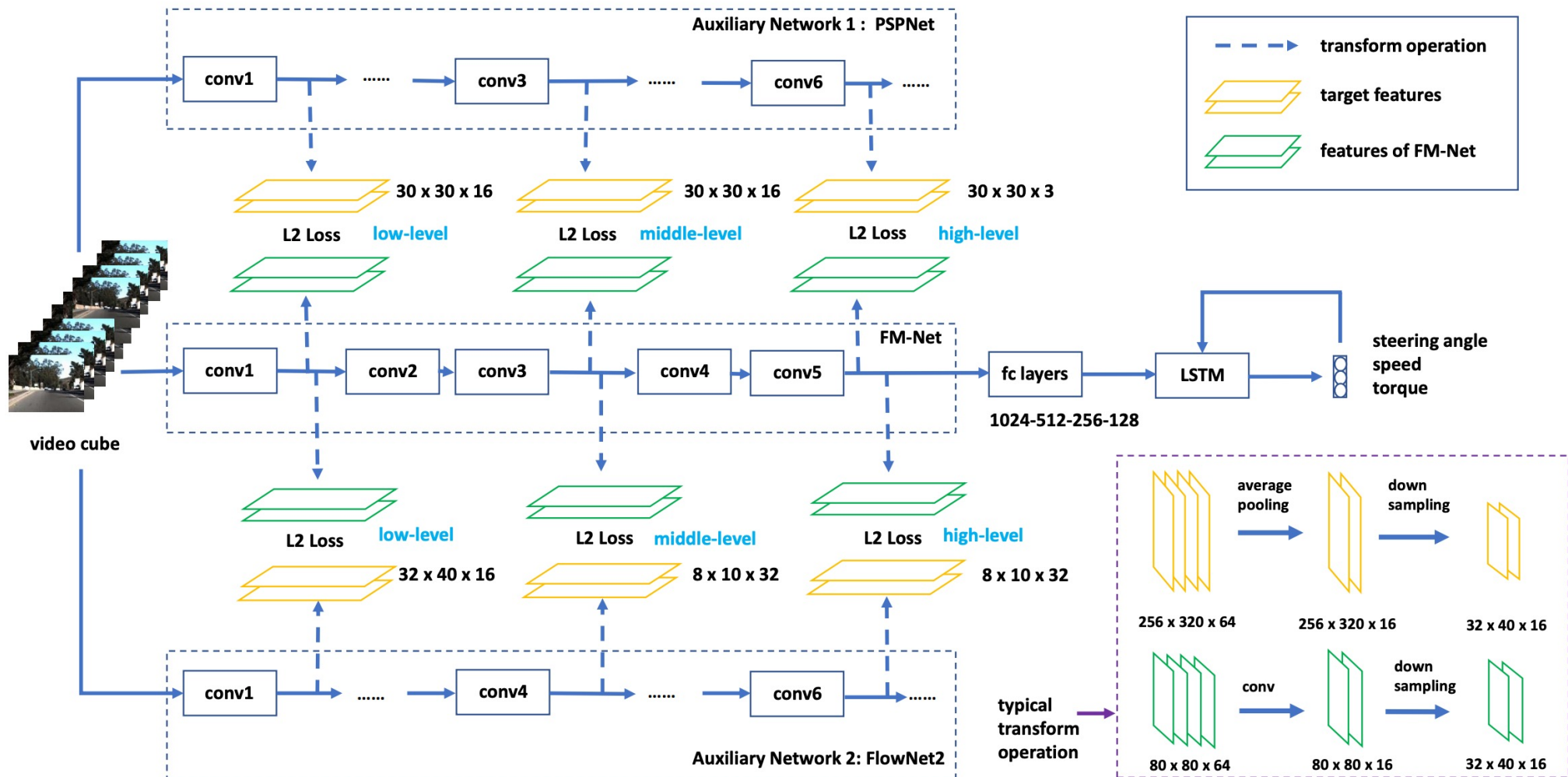
- Busy traffic
- Challenging light conditions

Application - transfer hidden activation

- Heterogeneous auxiliary networks feature mimicking - borrow features from other networks
- There are many strong and state-of-the-art networks such as PSPNet (Zhao et al. 2017) for segmentation, and FlowNet2 (Ilg et al. 2017) for optical flow estimation.
- Applying these networks on the target data can generate features that are highly indicative of the final prediction of steering angles.



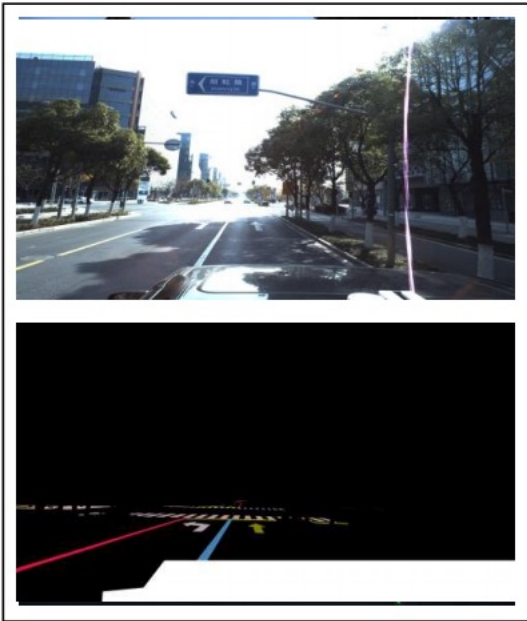
Application - transfer hidden activation



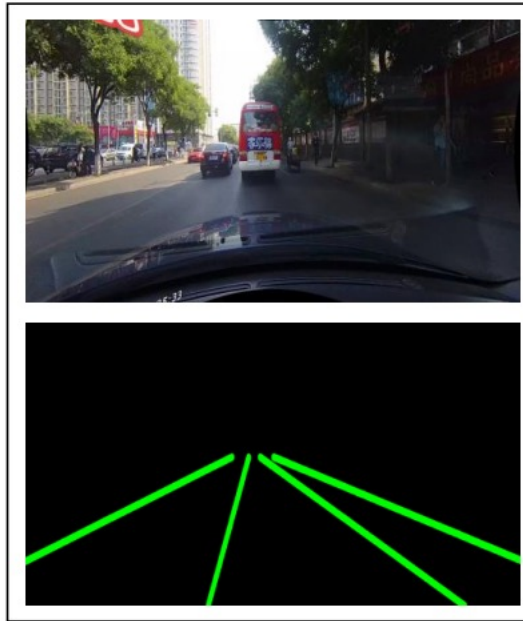
Application – transfer relation

- Road marking segmentation

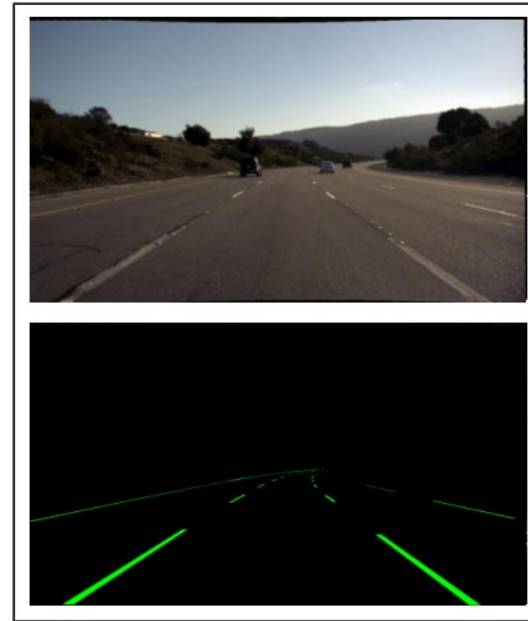
ApolloScape



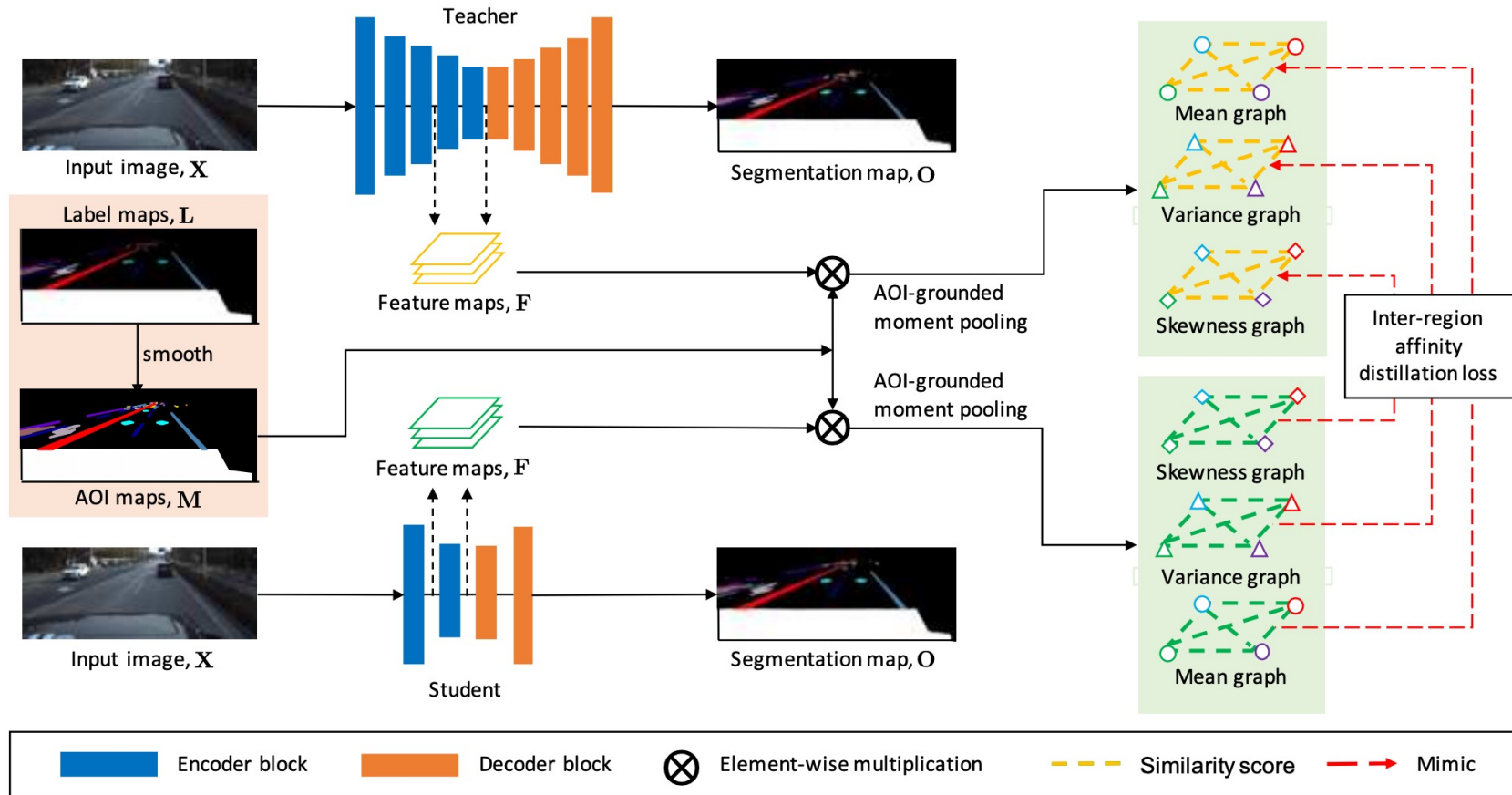
CULane



LLAMAS



Application – transfer relation



Inter-region affinity distillation

Each region is a part of a deep feature map, while each node in the graph denotes the feature distribution statistics of each region.

Each pair of nodes are connected by an edge representing their similarity in terms of feature distribution.

Given the same input image, the student network and the teacher network will both produce their corresponding affinity graph.

Through graph matching, a distillation loss on graph consistency is generated to update the student network

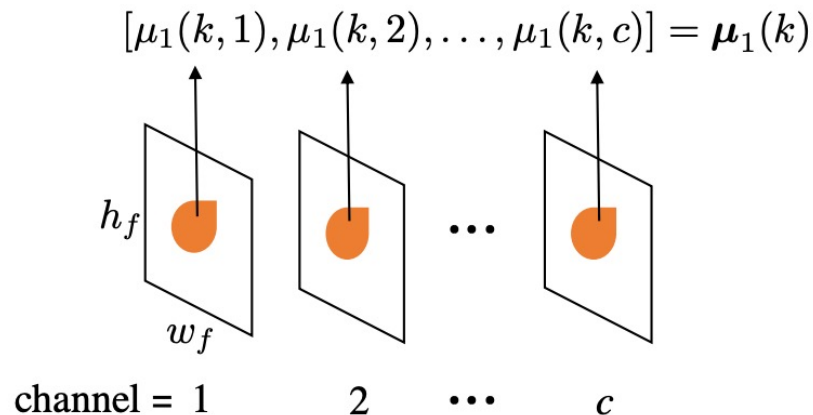
Application – transfer relation

- Moment pooling

$$\text{Mean} \quad \mu_1(k) = \frac{1}{|\mathbf{M}(:, :, k)|} \sum_{i=1}^{h_f} \sum_{j=1}^{w_f} \mathbf{M}(i, j, k) \mathbf{F}(i, j),$$

$$\text{Variance} \quad \mu_2(k) = \frac{1}{|\mathbf{M}(:, :, k)|} \sum_{i=1}^{h_f} \sum_{j=1}^{w_f} (\mathbf{M}(i, j, k) \mathbf{F}(i, j) - \mu_1(k))^2,$$

$$\text{Skewness} \quad \mu_3(k) = \frac{1}{|\mathbf{M}(:, :, k)|} \sum_{i=1}^{h_f} \sum_{j=1}^{w_f} \left(\frac{\mathbf{M}(i, j, k) \mathbf{F}(i, j) - \mu_1(k)}{\mu_2(k)} \right)^3,$$



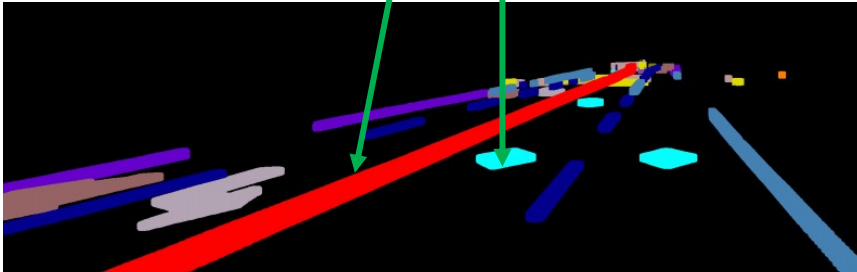
$$\mathbf{C}(k_1, k_2, r) = \frac{\boldsymbol{\mu}_r(k_1)^\top \boldsymbol{\mu}_r(k_2)}{\|\boldsymbol{\mu}_r(k_1)\|_2 \|\boldsymbol{\mu}_r(k_2)\|_2}, r \in \{1, 2, 3\}.$$

moments

Application – transfer relation

- Cosine similarity of the moment vectors of class k_1 and class k_2

$$\mathbf{C}(k_1, k_2, r) = \frac{\boldsymbol{\mu}_r(k_1)^\top \boldsymbol{\mu}_r(k_2)}{\|\boldsymbol{\mu}_r(k_1)\|_2 \|\boldsymbol{\mu}_r(k_2)\|_2}, r \in \{1, 2, 3\}. \quad \text{moments}$$



- The inter-region affinity distillation loss

$$\mathcal{L}_m(\mathbf{C}_S, \mathbf{C}_T) = \frac{1}{3n^2} \sum_{r=1}^3 \sum_{k_1=1}^n \sum_{k_2=1}^n \|\mathbf{C}_S(k_1, k_2, r) - \mathbf{C}_T(k_1, k_2, r)\|_2^2.$$